

An MILP-Based Solution Scheme for Factored Markov Decision Processes

Huikang Liu^{*1}, Wolfram Wiesemann², and Man-Chung Yue³

¹*Antai College of Economics and Management Shanghai Jiao Tong University Xuhui Campus, Shanghai, China, hkl1u@sjtu.edu.cn*

²*Imperial College Business School, London, United Kingdom, ww@imperial.ac.uk*

³*Department of Applied Mathematics, The Hong Kong Polytechnic University, Hung Hom, Hong Kong, manchung.yue@polyu.edu.hk*

May 27, 2026

Abstract

Factored Markov decision processes (MDPs) are a prominent paradigm within the artificial intelligence community for modeling and solving large-scale MDPs whose rewards and dynamics decompose into smaller, loosely interacting components. Through the use of value function approximations, dynamic Bayesian networks and context-specific independence, factored MDPs can achieve an exponential reduction in the state space of an MDP and thus scale to problem sizes that are beyond the reach of classical MDP algorithms. However, factored MDPs are typically solved using custom-designed algorithms that can require meticulous implementations and considerable fine-tuning. In this paper, we propose a mathematical programming approach to solving factored MDPs. Unlike existing solution schemes, our approach leverages off-the-shelf solvers, which enables streamlined implementation and maintenance. Moreover, we exploit the factored structure in both state and action spaces, and we employ feature representations to unify existing methods while taking advantage of context-specific independence in problem classes that other approaches cannot solve efficiently. To further enhance scalability, we introduce a feature learning scheme that automatically identifies informative features and a dynamic basis

^{*}Corresponding author.

approximation scheme that adaptively refines our value function approximations. Our numerical experiments demonstrate the potential of our approach.

Keywords: Factored Markov decision processes, Context-Specific Independence, Bayesian Networks, Feature Learning, Dynamic Basis Approximation.

1 Introduction

Dating back to the seminal work of Bellman (1952), Markov decision processes (MDPs) have emerged as one of the major paradigms to model, analyze and solve dynamic decision problems affected by uncertainty (Puterman, 1994; Bertsekas, 1995). Nevertheless, traditional MDPs are hampered by the *curse of dimensionality* since their characterization of a problem’s dynamics and reward structure tends to increase exponentially in the length of the problem’s description. In recent decades, various research communities have devised strategies to circumvent the unfavorable scalability of traditional MDPs through approximation, decomposition or a blend of both principles.

A large number of physical, biological and social systems possess a nearly decomposable, hierarchical structure (Simon, 1962), and the corresponding MDP representations—while large in scale—naturally embody aggregations of smaller components that evolve more or less independently. This structure is leveraged by *factored MDPs* (FMDPs), a methodology popularized predominantly within the artificial intelligence community. Instead of adopting the flat state space representation of traditional MDPs that enumerates all states sequentially (such as $s = 1, 2, \dots$), FMDPs characterize each state of an MDP via value assignments to state variables (such as “**velocity** = **high**, **acceleration** = **low**”), and these assignments evolve and contribute to the system’s rewards largely independently. This enables a compact representation of the stochastic evolution of FMDPs through dynamic Bayesian networks, which describe the one-step changes in the state variables as functions of a small number of ancestral state variables, as opposed to the entire state. FMDPs have found applications in, among others, resource allocation (Guestrin et al., 2002a; Dolgov and Durfee, 2006), recommender systems (Tavakol and Brefeld, 2014), power plant (Reyes et al., 2004, 2009) and micro grid management (Angelidakis and Chalkiadakis, 2014, 2015), manufacturing (Boutilier et al., 2000; Delgado et al., 2011), infrastructure design (Huang et al., 2017), traffic management (Delgado et al., 2011) and robot task planning (Boutilier et al., 2000). Factored representations are also employed in reinforcement learning (Sallans and Hinton, 2004; Degris et al., 2006; Osband

and Van Roy, 2014) and partially observable MDPs (Williams et al., 2005).

Perhaps due to its artificial intelligence heritage, the solution of FMDPs has largely relied upon hand-designed algorithms that iteratively manipulate decision trees, algebraic decision diagrams and/or rule systems representing the dynamics of the system as well as the value function (Boutilier et al., 1995; Hoey et al., 1999; Zhang and Poole, 1999). While there is no doubt that the resulting algorithms have significantly enhanced our comprehension of FMDPs, thereby facilitating numerous successful practical applications, these algorithms come with their own set of challenges. Most notably, they require considerable care during (re-)implementation, they tend not to exploit recent advancements in computational hardware (such as the parallelizability provided by contemporary multi-core architectures), and they demand considerable technical and domain-specific knowledge for fine-tuning their hyperparameters.

In this paper, we put forth a new framework that leverages mathematical programming to model and solve FMDPs. Instead of requiring the state evolution and reward structure to exhibit a sparse dependence on the current state, our model imposes a sparse dependence on a mixed-integer linear programming (MILP) representable feature vector derived from the current state and action. Compared to the existing literature on FMDPs, our MILP feature-based representation enjoys at least three advantages: *(i)* it enables us to model richer classes of dependencies since the feature vector may exhibit a complex dependence on the entire state without impacting the scalability of our approach; *(ii)* it allows us to achieve an exponential reduction not only in the state space but also in the action space; and *(iii)* it unifies and significantly extends the mainstream approaches for capturing context-specific independence (which we elaborate upon later). To solve the resulting FMDP formulation, we combine a well-known approximate linear programming (LP) formulation for traditional MDPs, which replaces the value function—whose tabular representation has exponentially many entries—with a linear combination of basis functions, with a novel cutting plane algorithm that identifies violated constraints through the solution of an MILP (rather than sequentially probing exponentially many constraints). We investigate the numerical performance of our algorithms in several case studies, which also compare them against alternative approaches from the literature.

The main contributions of this paper may be summarized as follows.

- (i)* We propose a new framework for FMDPs that is centered around MILP-representable fea-

tures. Our methodology allows us to exploit factored structure in larger classes of MDPs, and it enables us to achieve an exponential reduction in both the state and the action spaces.

- (ii) We show how an FMDP in our framework can be solved via an MILP-based cutting plane approach. In contrast to previous work, our approach relies on off-the-shelf solvers and avoids the need for hand-designed solution schemes.
- (iii) We discuss extensions of our framework that iteratively learn suitable feature representations and that dynamically extend the basis approximation of our value function.

More broadly, we see this work as a step towards the ambitious goal of integrating the insights of two influential yet distinct views on dynamic decision-making under uncertainty: the artificial intelligence approach, which has been pivotal in introducing, examining and leveraging structure in MDPs, and the mathematical programming approach, which has been exceptionally successful in solving MDPs. The source code of our framework, together with all experimental results, is made available open-source as part of this work to facilitate reuse as well as computational comparisons.¹

We are not the first ones to study FMDPs through the lens of mathematical programming. In a series of papers, Guestrin et al. (2001a,b, 2002b, 2003) and Schuurmans and Patrascu (2001) devise compact LP formulations for an approximate value iteration and an approximate policy iteration as well as an approximate linear programming approach for FMDPs. The authors conclude that their approximate linear programming approach is superior as it exhibits better scaling while at the same time requiring less stringent assumptions about the structure of the problem. In contrast to our cutting plane scheme, their approach relies on non-serial dynamic programming (Bertele and Brioschi, 1972) to equivalently replace an exponential number of constraints with a smaller, hopefully polynomial, number. Computing the smallest number of equivalent constraints is known to be NP-hard, and there are instances for which no equivalent reformulation with a small number of constraints exists. In practice, problem-specific heuristics are employed to generate small equivalent representations without performance guarantees. The non-serial dynamic programming approach differs from our method in at least two aspects: (i) it does not seem to accommodate MILP-based feature representations, which provide us with significant additional modelling flexibility, and (ii) it cannot leverage factored structure in the action space. We compare our cutting plane scheme with the non-serial dynamic programming approach in our numerical experiments.

¹Anonymous GitHub repository: <https://anonymous.4open.science/r/Factored-MDPs-A0F0/>.

Beyond FMDPs, other frameworks have been proposed to alleviate the curse of dimensionality of MDPs. Approximate dynamic programming (ADP), for example, offers a very general methodology to solve large-scale MDPs by approximating the value function through a linear combination of basis functions (Bertsekas and Tsitsiklis, 1996; Powell, 2011). In contrast to FMDPs, ADP does not rely on any specific problem structure. However, this generality implies that the associated solution approaches either require stronger assumptions (such as access to the stationary distribution under the unknown optimal policy) or provide weaker guarantees (such as asymptotic rather than finite convergence). More recently, Chen et al. (2018) proposed a primal-dual approach that solves generic large-scale MDPs by simultaneously approximating the primal and dual LP formulation via basis functions. The authors solve the resulting saddle-point problem via a sampling scheme. We finally note the related stream of literature on weakly coupled dynamic programming, which studies systems similar to FMDPs, but where the one-step transitions and rewards of the constituent MDPs are completely independent once a resource allocation decision has been taken (Hawkins, 2003; Adelman and Mersereau, 2008; Bertsimas and Mišić, 2016). This assumption, which has been relaxed to a joint dependence on a shared exogenous signal in the recent work of Brown and Zhang (2022), makes weakly coupled dynamic programming less expressive than FMDPs. We will compare ourselves against ADP, the primal-dual approach of Chen et al. (2018) and the weakly coupled dynamic programming approach of Brown and Zhang (2022) in our numerical experiments.

The remainder of the paper unfolds as follows. Sections 2 and 3 define FMDPs as well as sparse representations via feature vectors, respectively. Section 4 develops our MILP-based cutting plane solution scheme. Section 5 proposes extensions of our framework that iteratively learn feature representations and dynamically refine the basis approximation of the FMDP’s value function. We close with numerical experiments in Section 6. Supplementary material and all proofs are provided in an electronic companion to maintain the conciseness of the main paper.

Notation. We define $\mathbb{B} = \{0, 1\}$ and let $[n] = \{1, \dots, n\}$ for any $n \in \mathbb{N}$. We denote by $\mathbf{1}$ the vector of all ones and by $\mathbf{e}_i$ the i -th canonical basis vector, respectively; in both cases, the dimension will be clear from the context. For a finite set $\mathcal{X} = [n]$, we denote by $\Delta(\mathcal{X})$ the set of all probability distributions over $\mathcal{X}$, which we interpret either as functions $\Delta = \{(p : \mathcal{X} \rightarrow [0, 1]) : \sum_{x \in \mathcal{X}} p(x) = 1\}$ or as vectors $\Delta = \{p \in [0, 1]^n : \mathbf{1}^\top p = 1\}$, depending on the context. We define $\mathbf{1}[\mathcal{E}]$ as the indicator function that takes the value 1 (0) if the condition $\mathcal{E}$ is (not) satisfied.

2 Factored Markov Decision Processes

A (non-factored) *Markov decision process* (MDP) is defined by the tuple $(\mathcal{S}, \mathcal{A}, q, p, r, \gamma)$, where $\mathcal{S}$ and $\mathcal{A}$ represent the finite state and action spaces, $q \in \Delta(\mathcal{S})$ describes the distribution of the initial state, the transition probabilities $p : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ characterize the stochastic one-step evolution of the system, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, and $\gamma \in (0, 1)$ is the discount factor. MDPs do not capture any intrinsic structure that an application domain may possess, and they thus often scale exponentially in a natural representation of the problem.

Example 1 (Predictive Maintenance). *Consider a maintenance problem where multiple pieces of equipment are operated in sequence, and where the condition of upstream equipment impacts the deterioration rate of downstream equipment. This is the case, for example, (i) in assembly lines where a poor calibration of the initial cutting step results in subtle defects that can impact the subsequent welding and soldering steps, (ii) in the oil and gas industry, where poorly maintained drilling machines result in sediments and other impurities that lead to a faster wear and tear of downstream equipment such as pipelines, pumps, and processing units, and (iii) in power plants, where poorly maintained boilers can generate steam at an incorrect pressure or temperature that causes excessive wear and tear at downstream equipment such as turbines.*

Assume that 10 machines are operated in sequence, each of which is in one of 10 states ranging from 1 (malfunction) to 10 (optimal performance). In each period, exactly 2 machines should undergo maintenance. The problem can be modeled as a non-factored MDP with 10^{10} states and $\binom{10}{2} = 45$ actions, which requires the consideration of $10^{10} \cdot 45 \cdot 10^{10} = 4.5 \cdot 10^{21}$ transitions probabilities.

Oftentimes, we can achieve an exponential reduction in the representation of a problem by adopting a factored representation of the state space (Boutilier et al., 2000; Guestrin et al., 2003).

Definition 1 (Factored MDP). *A factored MDP (FMDP) is defined by the tuple $(\mathcal{S}, \mathcal{A}, q, p, r, \gamma)$, where the finite state space $\mathcal{S} = \times_{n \in [N]} \mathcal{S}_n$ constitutes a product of binary sub-state spaces $\mathcal{S}_n \subseteq \mathbb{B}^{S_n}$, the finite action space is $\mathcal{A} \subseteq \mathbb{B}^A$, the initial distribution is $q = \{q_n\}_{n \in [N]}$ with $q_n \in \Delta(\mathcal{S}_n)$, the transition kernel is $p = \{p_n\}_{n \in [N]}$ with $p_n : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S}_n)$, the reward function is $r = \{r_j\}_{j \in [J]}$ with $r_j : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, and the discount factor is $\gamma \in (0, 1)$.*

Unlike non-factored MDPs, the state space $\mathcal{S}$ of an FMDP is the Cartesian product of N sub-state spaces $\mathcal{S}_n$, $n \in [N]$, where each $\mathcal{S}_n$ is a set of S_n -dimensional binary vectors. For illustration,

an FMDP tracking the state variables “velocity” and “acceleration” could have $N = 2$ sub-state spaces with $S_n = 2$, $\mathcal{S}_n = \{(0, 0), (0, 1), (1, 0)\} \subset \mathbb{B}^2$, encoding “low”, “medium”, and “high” as $(0, 0)$, $(0, 1)$, and $(1, 0)$, respectively. Similarly, we assume that the action space is represented as a subset of $\mathbb{B}^A$; in many practical applications $A \propto N$, and action vectors admit a natural interpretation as stacked sub-vectors, one for each sub-state space. The restriction to binary sub-state and action spaces in Definition 1 is standard in the FMDP literature, and it is taken in view of the solution methods that we develop later. That said, all of our theory readily extends to general finite sub-state and action spaces. We assume that the one-step rewards decompose as $r(s, a) = \sum_{j \in [J]} r_j(s, a)$. This representation will later allow us to efficiently solve problems in which $r(s, a)$ depends densely on (s, a) (a dependence that—as we show—is unfavorable), provided it can be written as a sum of components $r_j(s, a)$, each with sparse dependence on (s, a) . The assumption that the components r_j depend only on the current state and action, and not on the subsequent state, entails no loss of generality and is made solely to streamline notation in our results.

A deterministic and stationary (*i.e.*, time-invariant) *policy* $\pi : \mathcal{S} \rightarrow \mathcal{A}$ for an FMDP assigns an action to each state. Under the policy π , the FMDP evolves as follows. The initial state $\tilde{s}^0$ is random and satisfies $\tilde{s}^0 = s$, where $s = (s_1, \dots, s_N) \in \mathcal{S}$, with probability $\prod_{n \in [N]} q_n(s_n)$. If the random state $\tilde{s}^t$ of the FMDP at time $t \in \mathbb{N}_0$ satisfies $\tilde{s}^t = s$, $s \in \mathcal{S}$, then the action $\pi(s)$ is taken and the system transitions to the new state $\tilde{s}^{t+1} = s'$, $s' = (s'_1, \dots, s'_N) \in \mathcal{S}$, with probability $\prod_{n \in [N]} p_n(s'_n | s, \pi(s))$. In other words, both the initial and the transition probabilities of an FMDP multiply across sub-states. For the transition probabilities, this implies that conditional on the current state and the selected action, each sub-state evolves independently. We are interested in policies π^* that maximize the *expected total discounted reward* $\mathbb{E}[\sum_{t \in \mathbb{N}_0} \gamma^t \cdot r(\tilde{s}^t, \pi(\tilde{s}^t)) | \tilde{s}^0 \sim q]$, where the stochastic state evolution $\{\tilde{s}^t\}_{t \in \mathbb{N}_0}$ is governed by q , p and π as described above, and where the one-step rewards $r(s, a)$ constitute sums of the reward components $r_j(s, a)$, $j \in [J]$. It is well-known that there always exists a deterministic and stationary policy that maximizes the expected total discounted reward among all (possibly non-stationary and/or randomized) policies, see, *e.g.*, Puterman (1994, Theorem 6.2.7).

Example 2 (Predictive Maintenance, Cont’d). *Assume that in Example 1, different machines deteriorate independently of another, conditional on the current state and action. We can model this problem as an FMDP with 10 sub-state spaces, one for each machine’s condition, and 45*

actions, thus requiring the consideration of $10 \cdot 10^{10} \cdot 45 \cdot 10 = 4.5 \cdot 10^{13}$ transition probabilities.

The representation of an FMDP can be compressed further when the rewards and transition probabilities only depend on parts of the current state. To formalize this idea, we define the *scope* of a function $f : \mathcal{X} \rightarrow \mathbb{R}$ with $\mathcal{X} \subseteq \mathbb{B}^I$ as $\mathfrak{s}[f] = [I] \setminus \{i \in [I] : x_{-i} = x'_{-i} \Rightarrow f(x) = f(x') \ \forall x, x' \in \mathcal{X}\}$. Thus, the scope of f comprises the indices of those components of f 's input that can impact f 's value. We informally say that f has a *low scope* if $|\mathfrak{s}[f]| \ll I$, that is, when $|\mathfrak{s}[f]|$ is much smaller than I . More formally, a sequence of functions $\{f^j\}_{j \in \mathbb{N}}$, $f^j : \mathcal{X}^j \rightarrow \mathbb{R}$ with $\mathcal{X}^j \subseteq \mathbb{B}^{I_j}$ and I_j monotonically increasing, has a low scope if $|\mathfrak{s}[f^j]|$ is bounded from above by a logarithm of I_j as j grows large. We also define $\mathfrak{S}[f] = \{\sum_{i \in [I]} \mathbf{1}[i \in \mathfrak{s}[f]] \cdot x_i \cdot \mathbf{e}_i : x \in \mathcal{X}\}$, where $\mathbf{1}[i \in \mathfrak{s}[f]] \cdot x_i \cdot \mathbf{e}_i$ is a multiplication between scalars and $\mathbf{e}_i \in \mathbb{R}^I$, as the set of all sub-vectors with components $i \in \mathfrak{s}[f]$, padded with zeros to form a valid input to f . (If $\mathcal{X}$ does not contain some of these padded vectors due to the presence of the zeros, then the domain of f can be extended in the obvious way to accommodate for those inputs.) Note that $|\mathfrak{S}[f]| \ll |\mathcal{X}|$ whenever f has a low scope. FMDPs are particularly powerful when the reward components r_j and the transition probabilities p_n exhibit a low scope.

Example 3 (Predictive Maintenance, Cont'd). *Assume that in Example 2, the transition probabilities of each machine only depend on the current state of that machine, whether or not that machine is being repaired, as well as the current state of its immediate predecessor in the sequence (if existent). The resulting FMDP requires the consideration of $10 \cdot 2 \cdot 10 = 200$ transition probabilities for the first machine as well as $(10 \cdot 10) \cdot 2 \cdot 10 = 2,000$ transition probabilities for each subsequent machine, thus reducing the storage requirements to 18,200 transition probabilities.*

Informally, the transition probabilities in Example 3 exhibit a low scope since each p_n depends only on the current state of the machine n as well as at most one predecessor machine, as opposed to the current state of all machines. In view of our formal definition, consider a family of maintenance problems where the number N of machines varies. In that setting, each of the transition probabilities p_n would still only depend on the current state of at most two machines, that is, its scope would remain constant when the size of the problem grows.

So far, we have focused on exploiting a low scope in the transition probabilities. Analogously, we can exploit a low scope in the reward. We will see in Section 4 that the aggregate reward function $r(s, a)$ does not need to have low scope; it is sufficient for r to decompose into a sum of low-scope components $r_j : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, which motivates their introduction in Definition 1. In the

context of our maintenance problem, for example, it is natural to assume that the rewards in each period constitute a sum of machine-specific rewards $r_j(s, a)$, $j \in [N]$, that are determined by each machine’s state s_j .

In addition to a low-scope structure, FMDPs can exploit context-specific independence. Section 3 describes this concept and introduces an MILP-based feature representation that unifies and extends existing approaches for context-specific independence. Afterwards, Section 4 develops and analyzes a cutting plane scheme to solve FMDPs with MILP-based feature representations.

3 MILP-Based Feature Representation

So far, our parsimonious representation of MDPs exploits (i) a *factored structure*, under which the one-step transitions are characterized by the product $\prod_{n \in [N]} p_n(s'_n | s, a)$ of individual transition probabilities p_n for each sub-state $n \in [N]$, as well as (ii) a *low-scope structure*, under which the transition probabilities p_n and reward components r_j exhibit a low scope with respect to the current state $s \in \mathcal{S}$ of the system. Additionally, many application domains possess *context-specific independence*, under which each individual transition probability p_n and reward component r_j only depend on a small subset of its scope in any particular context (*i.e.*, the current state and selected action).

Example 4 (Predictive Maintenance, Cont’d). *Assume that in Example 3, the transition probabilities of each machine only depend on the current state of that machine, whether or not that machine is being repaired, as well as whether or not its immediate predecessor in the sequence is in a poor condition (state 1, 2 or 3). We can model this problem as an FMDP with context-specific independence that requires the storage of $10 \cdot 2 \cdot 10 = 200$ transition probabilities for the first machine as well as $(2 \cdot 10) \cdot 2 \cdot 10 = 400$ transition probabilities for each subsequent machine, thus reducing the consideration to 3,800 transition probabilities. Here, the reduction from 2,000 to 400 transition probabilities for the machines 2, $\dots$, 10 is achieved by distinguishing between the two contexts where the predecessor machine is in poor condition or not, rather than storing explicitly the transition probabilities for each of the 10 possible states of the predecessor machine.*

Traditionally, FMDPs capture context-specific independence via decision trees (Boutilier et al., 1995), algebraic decision diagrams (ADDs; Hoey et al. 1999) or rule systems (Zhang and Poole, 1999). Instead of adopting these methodologies, we propose to exploit context-specific independence

via MILP-based feature representations. More precisely, we express the dependence of the reward components r_j and transition probabilities p_n on both s and a via a *feature map* $\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{B}^F$ that maps state-action pairs (s, a) to vectors $\phi(s, a) \in \mathbb{B}^F$ of F binary features:

$$p_n(\cdot | s, a) = \psi_n(\phi(s, a)) \quad \text{and} \quad r_j(s, a) = \rho_j(\phi(s, a)) \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}, \quad (1a)$$

where $\psi_n : \mathbb{B}^F \rightarrow \Delta(\mathcal{S}_n)$, $n \in [N]$, and $\rho_j : \mathbb{B}^F \rightarrow \mathbb{R}$, $j \in [J]$, can be any functions as long as they exhibit low scopes with respect to their inputs $\varphi \in \{\phi(s, a) : (s, a) \in \mathcal{S} \times \mathcal{A}\}$. We require the feature map ϕ to have a mixed-integer linear representation of the form

$$\phi(s, a) = \varphi \iff (\varphi, \zeta; s, a) \in \mathcal{F} \text{ for some } \zeta \in \mathbb{R}^{F_l} \times \mathbb{B}^{F_b}, \quad (1b)$$

where $\mathcal{F}$ is a polyhedron described by F_c linear constraints and ζ is a vector of F_l continuous and F_b binary auxiliary variables. Note that the representation via feature maps does not restrict generality since ϕ can be chosen to be the identity map. In practice, however, we are interested in feature maps under which ψ_n and ρ_j exhibit a low scope. For the sake of simplicity and to support intuition, we henceforth commit a slight abuse of notation and write

$$p_n(\cdot | s, a) = p_n(\cdot | \phi(s, a)) \quad \text{and} \quad r_j(s, a) = r_j(\phi(s, a)) \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}$$

instead of ψ_n and ρ_j in (1a).

Example 5 (Predictive Maintenance, Cont'd). *Consider again the maintenance problem from Example 4. Representing the problem as an FMDDP, we can define the state space as $\mathcal{S} = \times_{n \in [10]} \mathcal{S}_n$, where each sub-state space $\mathcal{S}_n = \{s_n \in \mathbb{B}^{10} : \sum_{i \in [10]} s_{n,i} = 1\}$ adopts a unary description of the machine's state under which $s_{n,i} = 1$ if and only if machine n is in state $i \in [10]$. Likewise, we can define the action space as $\mathcal{A} = \{a \in \mathbb{B}^{10} : \sum_{n \in [10]} a_n = 2\}$ such that machine n is repaired if and only*

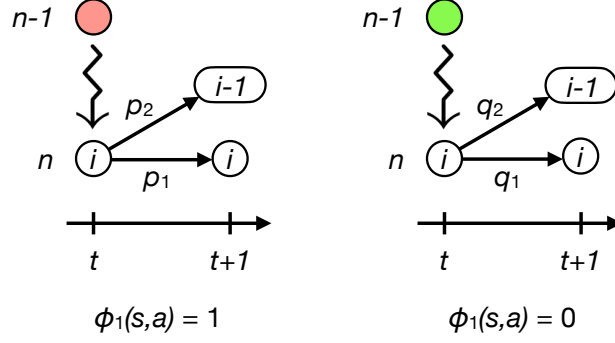


Figure 1. Transition structure for machine $n > 1$ in state $i > 1$ under no repair ($a_n = 0$) in Example 5. If the predecessor machine $n-1$ is in a poor state (left, $\phi_1(s, a) = 1$), machine n remains in state i with probability p_1 or deteriorates to state $i-1$ with probability p_2 . If the predecessor is not in a poor state (right, $\phi_1(s, a) = 0$), the corresponding probabilities are q_1 and q_2 .

if $a_n = 1$. Fix any machine $n > 1$, state $i \in [9]$ and action $a \in \mathcal{A}$ with $a_n = 0$, and assume that

$$p_n(s'_{n,i} = 1 \mid s, a) = \begin{cases} p_1 & \text{if } s_{n-1,j} = 1 \text{ for some } j \in [3] \text{ and } s_{n,i} = 1, \\ p_2 & \text{if } s_{n-1,j} = 1 \text{ for some } j \in [3] \text{ and } s_{n,i+1} = 1, \\ q_1 & \text{if } s_{n-1,j} = 0 \text{ for all } j \in [3] \text{ and } s_{n,i} = 1, \\ q_2 & \text{if } s_{n-1,j} = 0 \text{ for all } j \in [3] \text{ and } s_{n,i+1} = 1, \\ 0 & \text{otherwise.} \end{cases}$$

Here, p_i and q_i , $i = 1, 2$, are constants from the application domain that satisfy $p_1 + p_2 = 1$ and $q_1 + q_2 = 1$. We can represent these transition probabilities with the MILP-based features $\phi_1(s, a) = \mathbf{1}[s_{n-1,j} = 1 \text{ for some } j \in [3]]$, $\phi_2(s, a) = \mathbf{1}[s_{n,i} = 1]$ and $\phi_3(s, a) = \mathbf{1}[s_{n,i+1} = 1]$ by setting

$$p_n(s'_{n,i} = 1 \mid s, a) = \begin{cases} p_1 & \text{if } \phi_1(s, a) = 1 \text{ and } \phi_2(s, a) = 1, \\ p_2 & \text{if } \phi_1(s, a) = 1 \text{ and } \phi_3(s, a) = 1, \\ q_1 & \text{if } \phi_1(s, a) = 0 \text{ and } \phi_2(s, a) = 1, \\ q_2 & \text{if } \phi_1(s, a) = 0 \text{ and } \phi_3(s, a) = 1, \\ 0 & \text{otherwise.} \end{cases}$$

The transition dynamics are illustrated in Figure 1. The features admit the MILP representation

$$\begin{aligned}\phi_1(s, a) &\geq s_{n-1,j} \quad \forall j \in [3]; & \phi_1(s, a) &\leq s_{n-1,1} + s_{n-1,2} + s_{n-1,3}; \\ \phi_2(s, a) &= s_{n,i}; & \phi_3(s, a) &= s_{n,i+1}.\end{aligned}$$

Compared to trees, ADDs and rule systems, MILP-based feature representations offer various benefits: (i) while perhaps subjective, we would argue that they are easier to define, (ii) they can exploit parsimony in both the state and the action space, and (iii) they are at least as efficient and sometimes substantially more efficient in characterizing context-specific independence. The second and third advantages are illustrated in the E-Companion A.

Despite their versatility, even MILP-based feature representations can scale unfavorably in the worst case. This is shown by the next result, which provides an asymptotically tight characterization of the complexity of MILP-based feature representations in the context of our FMDPs.

Theorem 1 (Feature Complexity). *Any feature map $\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{B}^F$ affords a linear representation with at most $2F \cdot |\mathcal{S} \times \mathcal{A}|$ constraints, no auxiliary variables, and an encoding size polynomial in F , $|\mathcal{S} \times \mathcal{A}|$, and $\sum_{n \in [N]} S_n + A$. Furthermore, unless $P = NP$, there exists a family of feature maps ϕ for which any linear representation has an encoding size that is super-polynomial in $\log |\mathcal{S} \times \mathcal{A}|$.*

Recall that the encoding size of a representation is the total number of bits needed to specify all coefficients and right-hand sides in $\mathcal{F}$. Theorem 1 shows that any feature map admits a linear representation with encoding size polynomial in F , $|\mathcal{S} \times \mathcal{A}|$, and $\sum_{n \in [N]} S_n + A$, and that unless $P = NP$, this dependence on $|\mathcal{S} \times \mathcal{A}|$ cannot in general be reduced to polynomial in $\log |\mathcal{S} \times \mathcal{A}|$.

While MILP-based feature representations may scale exponentially in the worst case, compact representations often suffice for powerful features that record complex dependencies on s and a .

Proposition 1 (Feature Maps).

1. Sub-state selection (single). *The feature $\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n]$, where $\mathcal{S}'_n \subseteq \mathcal{S}_n$, can be represented using $F_l = |\mathcal{S}'_n|$ and $F_b = 0$ variables as well as $F_c = 1 + (1 + S_n)|\mathcal{S}'_n|$ constraints.*
2. Sub-state selection (universal). *The feature $\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n \quad \forall n \in \mathcal{N}]$, where $\mathcal{N} \subseteq [N]$ and $\mathcal{S}'_n \subseteq \mathcal{S}_n$, $n \in \mathcal{N}$, can be represented using $F_l = \sum_{n \in \mathcal{N}} |\mathcal{S}'_n|$ and $F_b = 0$ variables as well as $F_c = 1 + |\mathcal{N}| + \sum_{n \in \mathcal{N}} (1 + S_n) \cdot |\mathcal{S}'_n|$ constraints.*
3. Sub-state selection (existential). *The feature $\phi(s, a) = \mathbf{1}[\exists n \in \mathcal{N} : s_n \in \mathcal{S}'_n]$, where $\mathcal{N} \subseteq [N]$ and $\mathcal{S}'_n \subseteq \mathcal{S}_n$, $n \in \mathcal{N}$, can be represented using $F_l = \sum_{n \in \mathcal{N}} |\mathcal{S}_n \setminus \mathcal{S}'_n|$ and $F_b = 0$ variables as well as $F_c = 1 + |\mathcal{N}| + \sum_{n \in \mathcal{N}} (1 + S_n) \cdot |\mathcal{S}_n \setminus \mathcal{S}'_n|$ constraints.*

4. Sub-state selection (lower bound). *The feature $\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n \text{ for at least } \nu \text{ different } n \in \mathcal{N}]$, where $\mathcal{N} \subseteq [N]$ and $\mathcal{S}'_n \subseteq \mathcal{S}_n$, $n \in \mathcal{N}$, can be represented using $F_l = \sum_{n \in \mathcal{N}} |\mathcal{S}'_n|$ and $F_b = 1$ variables as well as $F_c = 2 + \sum_{n \in \mathcal{N}} (1 + S_n) \cdot |\mathcal{S}'_n|$ constraints.*
5. Sub-state selection (upper bound). *The feature $\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n \text{ for at most } \nu \text{ different } n \in \mathcal{N}]$, where $\mathcal{N} \subseteq [N]$ and $\mathcal{S}'_n \subseteq \mathcal{S}_n$, $n \in \mathcal{N}$, can be represented using $F_l = \sum_{n \in \mathcal{N}} |\mathcal{S}'_n|$ and $F_b = 1$ variables as well as $F_c = 2 + \sum_{n \in \mathcal{N}} (1 + S_n) \cdot |\mathcal{S}'_n|$ constraints.*

Proposition 1 can be readily extended to analogous features for the selected action $a \in \mathcal{A}$. For the sake of brevity, we omit the (largely similar) statement. We close this subsection with a formal definition of the FMDPs that we will study in the remainder of this paper.

Definition 2 (Featured-Based Factored MDP). *A feature-based factored MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \phi, q, p, r, \gamma)$, where $\mathcal{S}$, $\mathcal{A}$, q and γ coincide with their counterparts from Definition 1, and ϕ , p and r are defined as in Equation (1).*

We will use the abbreviation FMDP to refer to both FMDPs (*cf.* Definition 1) and feature-based FMDPs; the context will make it clear which definition we are referring to.

4 Cutting Plane Solution Scheme

Exploiting the factored and low-scope structure as well as the context-specific independence that is often present in practical applications allows us to derive compact representations of MDPs. We next develop an iterative algorithm that computes approximately optimal policies for MDPs described by those representations.

Optimal policies for MDPs are typically computed via value iteration, (modified) policy iteration or linear programming (LP), see §6.3–6.5 and §6.9 of Puterman (1994). In this paper, we use the LP approach to determine approximately optimal policies for FMDPs. Our choice is motivated by the observation of Guestrin et al. (2003) that the LP approach tends to outperform both value and policy iteration for FMDPs. To this end, consider the following LP, which injects the factored

representation of FMDPs into the standard LP model for MDPs:

$$\begin{aligned}
& \underset{v}{\text{minimize}} && \sum_{s \in \mathcal{S}} \left[\prod_{n \in [N]} q_n(s_n) \right] v(s) \\
& \text{subject to} && v(s) \geq \sum_{j \in [J]} r_j(\phi(s, a)) + \gamma \sum_{s' \in \mathcal{S}} \left[\prod_{n \in [N]} p_n(s'_n | \phi(s, a)) \right] v(s') \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A} \quad (2) \\
& && v : \mathcal{S} \rightarrow \mathbb{R}
\end{aligned}$$

The decision variable v in this problem represents the value function of the FMDP; at optimality, $v(s)$ records the expected total discounted reward if the system starts in state $s \in \mathcal{S}$ and is operated under an optimal policy. Note that despite the factored structure of FMDPs, the optimal value function does not possess a factored representation in general. The constraints ensure that the value function v weakly overestimates the expected total discounted reward associated with any state $s \in \mathcal{S}$ under any possible policy, while the objective function minimizes the expected total discounted reward under the initial distribution q . Whenever $q_n(s_n) > 0$ for all $s \in \mathcal{S}_n$ and $n \in [N]$, Problem (2) has a unique optimal solution that coincides with the fixed point of the Bellman operator for the FMDP (Puterman, 1994, Theorem 6.2.2 and Section 6.9.1). Irrespective of whether Problem (2) has a unique solution, any optimal solution v^* to the problem allows us to recover an optimal policy π^* for the FMDP via the *greedy policy*

$$\pi^*(s) \in \arg \max_{a \in \mathcal{A}} \left\{ \sum_{j \in [J]} r_j(\phi(s, a)) + \gamma \sum_{s' \in \mathcal{S}} \left[\prod_{n \in [N]} p_n(s'_n | \phi(s, a)) \right] v^*(s') \right\} \quad \forall s \in \mathcal{S},$$

see Puterman (1994, Theorem 6.2.7). Any optimal policy constructed this way attains an expected total discounted reward that coincides with the optimal objective value of Problem (2).

Problem (2) appears to be difficult to solve as it comprises exponentially many decision variables and constraints as well as sums over exponentially many terms. In fact, we next show that in contrast to ordinary MDPs, which can be solved in polynomial time, Problem (2) is EXPTIME-hard due to its ability to represent complex problems in a compact way. In some sense, this hardness result reflects a tension inherent in the FMDP paradigm: while the standard LP for (non-factored) MDPs can be solved in strongly polynomial time since it scales with $|\mathcal{S}| \times |\mathcal{A}|$, FMDPs allow for problem descriptions that are logarithmic in $|\mathcal{S}| \times |\mathcal{A}|$, and the computational tractability of the

LP cannot keep pace with this exponential compression.

Theorem 2. *Computing the optimal expected total discounted reward of an FMDP is EXPTIME-hard even when the feature map ϕ is the identity and all transition probabilities and reward components have scopes that are logarithmic in N .*

In the following, we simplify Problem (2) in two steps. We first replace the value function v , which has exponentially many components, with a linearly weighted approximation, which will also allow us to eliminate the exponentially-sized sums in (2). After that, we introduce the constraints of Problem (2) iteratively through a cutting plane scheme.

In view of the first step, we approximate the value function $v : \mathcal{S} \rightarrow \mathbb{R}$ in Problem (2) by

$$v'(s) = \sum_{k \in [K]} w_k \cdot \nu_k(s),$$

where $\nu_k : \mathcal{S} \rightarrow \mathbb{R}$ are pre-selected linearly independent basis functions and $w_k \in \mathbb{R}$ become the new decision variables in Problem (2), $k \in [K]$. To ensure tractability of our solution scheme, we require the *block-wise scope* of each basis function ν_k , $\bar{s}[\nu_k] = [N] \setminus \{n \in [N] : s_{-n} = s'_{-n} \Rightarrow \nu_k(s) = \nu_k(s') \ \forall s, s' \in \mathcal{S}\}$, $k \in [K]$, to be low. In contrast to our earlier definition of the scope $\mathfrak{s}$, which considers the individual binary function arguments, the block-wise scope $\bar{s}$ considers the sub-states of the input (which consist of $S_1, S_2, \dots, S_N$ binary components each) as atomic units. In other words, our approximation v' of the optimal value function v in Problem (2) should constitute a linear combination of linearly independent basis functions ν_k that each depend only on a small number of sub-states. Unless the FMDP has a very specific structure, our value function approximation is not guaranteed to be exact, and replacing v with v' will reduce the feasible region in Problem (2).

Example 6 (Value Function Approximation). *Consider an FMDP with $N \in \mathbb{N}$, $\mathcal{S}_n = \mathbb{B}$, $\mathcal{A} = \emptyset$ (a factored Markov reward process), $q_n(0) = q_n(1) = 1/2$, $p_n(s'_n \mid s, \emptyset) = 1/2$ for $s'_n \in \mathcal{S}_n$ and $s \in \mathcal{S}$, $r(s, \emptyset) = \sum_{n \in [N]} r_n(s, \emptyset)$ with $r_n(s, \emptyset) = s_n$, $n \in [N]$, as well as $\gamma = 0.9$. In this case, the value function approximation $\nu_k(s) = s_k$ for $k \in [N]$ and $\nu_K(s) \equiv 1$ for $K = N + 1$ is exact—despite only $N + 1$ basis functions approximating a state space of dimension $|\mathcal{S}| = 2^N$ —since the FMDP perfectly decouples into N separate MDPs. On the other hand, our value function approximation is no longer exact when we replace the rewards with $r(s, \emptyset) = r_1(s, \emptyset)$ and $r_1(s, \emptyset) = \mathbf{1}[s_1 = s_2 = \dots = s_N]$, that is, if a reward is only received when all sub-states are equal. Likewise, our value function*

approximation becomes inexact if we replace the transition kernel with $p_n(1 - s_n \mid s, \emptyset) = N^{-1} \cdot \sum_{i \in [N]} s_i$, that is, if the switching probabilities increase with the number of ones in the current state.

In absence of domain knowledge, one may employ scope-1, scope-2 or scope-3 value function approximations. The scope-1 value functions comprise basis functions of the form $\nu_{(n, \sigma_n)} = \mathbf{1}[s_n = \sigma_n]$ for $\sigma_n \in \mathcal{S}_n$. The scope-2 value functions comprise basis functions of the form $\nu_{(n, \sigma_n, n', \sigma_{n'})} = \mathbf{1}[s_n = \sigma_n \text{ and } s_{n'} = \sigma_{n'}]$ for $n, n' \in [N]$, $n \neq n'$, and $\sigma_n \in \mathcal{S}_n$ and $\sigma_{n'} \in \mathcal{S}_{n'}$. The scope-3 value functions, finally, are constructed analogously for $\nu_{(n, \sigma_n, n', \sigma_{n'}, n'', \sigma_{n''})}$. In our numerical experiments, we will exploit structural intuition about the problems to consider a reduced set of basis functions.

Value function approximations are very common in ADP, see, *e.g.*, Schweitzer and Seidmann (1985), de Farias and Van Roy (2003) and §4.7.2 of Powell (2011). In contrast to the LP approach employed in that literature, which requires knowledge of the unknown optimal policy to sample constraints efficiently and which results in probabilistic guarantees, we can exploit the factored representation of FMDPs to design a cutting plane solution scheme that only relies on available information about the system and that results in deterministic guarantees.

Proposition 2. *Under our value function approximation, Problem (2) is equivalent to the LP*

$$\begin{aligned}
& \underset{w}{\text{minimize}} && \sum_{k \in [K]} w_k \sum_{s \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s) \prod_{n \in \bar{\mathcal{S}}[\nu_k]} q_n(s_n) \\
& \text{subject to} && \sum_{k \in [K]} w_k \cdot \nu_k(s) \geq \sum_{j \in [J]} r_j(\phi(s, a)) + \gamma \sum_{k \in [K]} w_k \sum_{s' \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s') \prod_{n \in \bar{\mathcal{S}}[\nu_k]} p_n(s'_n \mid \phi(s, a)) \\
& && \forall (s, a) \in \mathcal{S} \times \mathcal{A} \\
& && w \in \mathbb{R}^K.
\end{aligned} \tag{3}$$

Moreover, if there is a linear combination $w^1 \in \mathbb{R}^K$ of the basis functions that produces the all-one vector $\mathbf{1} \in \mathbb{R}^{|\mathcal{S}|}$, then Problem (3) attains its (finite) optimal value.

Note that the summations over $s \in \bar{\mathcal{S}}[\nu_k]$ in Problem (3) have moderate cardinalities whenever the basis functions ν_k have a low scope. Contrary to Problem (2), the optimal solution to Problem (3) may not be unique. Any optimal solution $w^\star$ to (3) allows us to recover an *approximately* optimal policy $\hat{\pi}$ for the FMDP via

$$\hat{\pi}(s) \in \arg \max_{a \in \mathcal{A}} \left\{ \sum_{j \in [J]} r_j(\phi(s, a)) + \gamma \sum_{k \in [K]} w_k^\star \sum_{s' \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s') \prod_{n \in \bar{\mathcal{S}}[\nu_k]} p_n(s'_n \mid \phi(s, a)) \right\} \tag{4}$$

Algorithm 1 Cutting Plane Scheme to Solve Problem (3)

- 1: **Input:** Initial constraint set $\mathcal{C} \subset \mathcal{S} \times \mathcal{A}$ and an optimality tolerance $\epsilon \geq 0$.
 - 2: Set constraint violation to $+\infty$.
 - 3: **while** constraint violation exceeds ϵ **do**
 - 4: **Master Problem:** Solve the variant of Problem (3) that only contains the constraints pertaining to $(s, a) \in \mathcal{C}$ and record the optimal weight vector w^* .
 - 5: **Subproblem:** Determine the constraint violation and the associated constraint
$$(s^*, a^*) \in \arg \max_{(s, a) \in \mathcal{S} \times \mathcal{A}} \left\{ \sum_{j \in [J]} r_j(\phi(s, a)) + \gamma \sum_{k \in [K]} w_k^* \sum_{s' \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s') \prod_{n \in \bar{\mathcal{S}}[\nu_k]} p_n(s'_n | \phi(s, a)) - \sum_{k \in [K]} w_k^* \nu_k(s) \right\}.$$
 - 6: If the constraint violation exceeds ϵ , add (s^*, a^*) to $\mathcal{C}$.
 - 7: **end while**
 - 8: **Output:** An approximately optimal solution $w^\epsilon = w^*$ to Problem (3).
-

for all $s \in \mathcal{S}$. Since the greedy policy $\hat{\pi}$ is constructed from an approximation of the optimal value function, it will no longer coincide with the optimal policy in general. We can estimate the expected total discounted reward generated by $\hat{\pi}$ via simulation: draw M i.i.d. initial states $\tilde{s}^0$ from the initial state distribution q , use those to simulate M i.i.d. trajectories of finite length T under p and $\hat{\pi}$, and record the discounted cumulative rewards collected along each trajectory. As $M, T \rightarrow \infty$, the strong law of large numbers ensures that the average of the discounted cumulative rewards approaches the expected total discounted reward of $\hat{\pi}$. Due to the suboptimality of $\hat{\pi}$, the simulation provides a (statistical) lower bound on the expected total discounted reward generated by the unknown optimal policy. On the other hand, the optimal value of Problem (3) bounds the expected total discounted reward generated by the unknown optimal policy from above since the value function approximation reduces the feasible region in Problem (3), and thus the optimal value of Problem (3) exceeds that of Problem (2). In summary, the solution of Problem (3) provides an implementable greedy policy as well as a bound on the suboptimality of that policy relative to the unknown optimal policy.

Determining $\hat{\pi}(s)$ according to Equation (4) appears to be difficult as it involves the consideration of exponentially many actions $a \in \mathcal{A}$. We will see below in Corollary 1, however, that $\hat{\pi}(s)$ can be evaluated through the solution of an MILP of moderate size. This allows for a compact implicit representation of $\hat{\pi}$ in terms of the weight vector $w \in \mathbb{R}^K$, as opposed to the value function v in Problem (2), which has exponentially many components.

For value function approximations such as our scope-1, scope-2 and scope-3 architectures de-

scribed above, Problem (3) has a small number of decision variables while the number of constraints remains large. To combat this issue, Algorithm 1 employs a cutting plane scheme to iteratively introduce only those constraints that are relevant for the solution of the problem. The algorithm assumes that the initial constraint set $\mathcal{C} \subset \mathcal{S} \times \mathcal{A}$ is large enough so that the restricted master problem is not unbounded, which can be readily verified.

The key characteristics of Algorithm 1 are summarized in the next result.

Theorem 3. *Algorithm 1 terminates in finite time. Let $F(w)$ and $F^\star$ denote the objective function value of $w \in \mathbb{R}^K$ in Problem (3) as well as the optimal value of (3), respectively. Let w^ϵ be the output of Algorithm 1, and assume that there is a linear combination $w^1 \in \mathbb{R}^K$ of the basis functions that produces the all-one vector $\mathbf{1} \in \mathbb{R}^{|\mathcal{S}|}$. Then $\hat{w} = w^\epsilon + \frac{\epsilon}{1-\gamma} \cdot w^1$ is feasible in (3) and satisfies*

$$F^\star - (1 - \gamma)^{-1} \cdot \epsilon \leq F(w^\epsilon) \leq F^\star \leq F(\hat{w}) \leq F^\star + (1 - \gamma)^{-1} \cdot \epsilon.$$

Theorem 3 implies that Algorithm 1 optimally solves Problem (3) when $\epsilon = 0$. The master problem of Algorithm 1 is an LP that can be solved efficiently. Without further structure, the subproblem of Algorithm 1 would require a complete enumeration of all state-action pairs $(s, a) \in [\mathcal{S} \times \mathcal{A}] \setminus \mathcal{C}$. The structure inherent in FMDPs allows us to determine $(s^\star, a^\star)$ much more efficiently.

Theorem 4. *A maximally violated constraint $(s^\star, a^\star) \in \mathcal{S} \times \mathcal{A}$ for fixed $w^\star \in \mathbb{R}^K$ in Problem (3) can be extracted from an optimal solution $(s^\star, a^\star, \varphi^\star, \zeta^\star, \eta^\star, \xi^\star, \beta^\star)$ to the MILP*

$$\begin{aligned} & \underset{\substack{s, a, \varphi, \zeta \\ \eta, \xi, \beta}}{\text{maximize}} && \sum_{j \in [J]} \sum_{f \in \mathfrak{S}[r_j]} r_j(f) \cdot \eta_{jf} + \gamma \sum_{k \in [K]} w_k^\star \sum_{f \in \mathfrak{S}[\bar{\nu}_k]} \bar{\nu}_k(f) \cdot \xi_{kf} - \sum_{k \in [K]} w_k^\star \sum_{s' \in \mathfrak{S}[\nu_k]} \nu_k(s') \cdot \beta_{ks'} \\ \text{subject to} && \eta_{jf} \in [0, 1], \quad 1 + \sum_{i \in \mathfrak{s}[r_j]} \frac{\varphi_i - f_i}{2f_i - 1} \leq \eta_{jf} \leq (2f_l - 1)\varphi_l + 1 - f_l && \forall j \in [J], \forall f \in \mathfrak{S}[r_j], \forall l \in \mathfrak{s}[r_j] \\ && \xi_{kf} \in [0, 1], \quad 1 + \sum_{i \in \mathfrak{s}[\bar{\nu}_k]} \frac{\varphi_i - f_i}{2f_i - 1} \leq \xi_{kf} \leq (2f_l - 1)\varphi_l + 1 - f_l && \forall k \in [K], \forall f \in \mathfrak{S}[\bar{\nu}_k], \forall l \in \mathfrak{s}[\bar{\nu}_k] \\ && \beta_{ks'} \in [0, 1], \quad 1 + \sum_{i \in \mathfrak{s}[\nu_k]} \frac{[s]_i - [s']_i}{2[s']_i - 1} \leq \beta_{ks'} \leq (2[s']_l - 1)[s]_l + 1 - [s']_l && \forall k \in [K], \forall s' \in \mathfrak{S}[\nu_k], \forall l \in \mathfrak{s}[\nu_k] \\ && (s, a) \in \mathcal{S} \times \mathcal{A}, \quad (\varphi, \zeta; s, a) \in \mathcal{F}, \quad \zeta \in \mathbb{R}^{F_l} \times \mathbb{B}^{F_b}, \end{aligned}$$

where the functions $\bar{\nu}_k : \mathbb{B}^F \rightarrow \mathbb{R}$, $k \in [K]$, are defined as

$$\bar{\nu}_k(\phi(s, a)) = \sum_{s' \in \mathfrak{S}[\nu_k]} \nu_k(s') \cdot \prod_{n \in \mathfrak{s}[\nu_k]} p_n(s'_n | \phi(s, a))$$

and $[s]_l$ refers to the l -th (binary) component of the binary vector s , $l = 1, \dots, \sum_{n \in [N]} S_n$.

One may wonder whether it is necessary to solve NP-hard MILPs in Algorithm 1 or whether maximally violated constraints can be identified in polynomial time. To this end, the next result shows that more efficient solution techniques than the one of Theorem 4 are unlikely to exist.

Proposition 3. *Identifying a maximally violated constraint in Algorithm 1 is strongly NP-complete even if all transition probabilities and reward components have scope 1.*

We can use a straightforward variation of the MILP in Theorem 4 to recover the greedy policy $\hat{\pi}$ characterized by Equation (4).

Corollary 1. *For fixed $s \in \mathcal{S}$ and $w^\star \in \mathbb{R}^K$, let $(a^\star, \varphi^\star, \zeta^\star, \eta^\star, \xi^\star)$ be optimal in the MILP*

$$\begin{aligned} & \underset{a, \varphi, \zeta, \eta, \xi}{\text{maximize}} && \sum_{j \in [J]} \sum_{f \in \mathfrak{S}[r_j]} r_j(f) \cdot \eta_{jf} + \gamma \sum_{k \in [K]} w_k^\star \sum_{f \in \mathfrak{S}[\bar{\nu}_k]} \bar{\nu}_k(f) \cdot \xi_{kf} \\ \text{subject to} &&& \eta_{jf} \in [0, 1], \quad 1 + \sum_{i \in \mathfrak{s}[r_j]} \frac{\varphi_i - f_i}{2f_i - 1} \leq \eta_{jf} \leq (2f_l - 1)\varphi_l + 1 - f_l \quad \forall j \in [J], \forall f \in \mathfrak{S}[r_j], \forall l \in \mathfrak{s}[r_j] \\ &&& \xi_{kf} \in [0, 1], \quad 1 + \sum_{i \in \mathfrak{s}[\bar{\nu}_k]} \frac{\varphi_i - f_i}{2f_i - 1} \leq \xi_{kf} \leq (2f_l - 1)\varphi_l + 1 - f_l \quad \forall k \in [K], \forall f \in \mathfrak{S}[\bar{\nu}_k], \forall l \in \mathfrak{s}[\bar{\nu}_k] \\ &&& a \in \mathcal{A}, \quad (\varphi, \zeta; s, a) \in \mathcal{F}, \quad \zeta \in \mathbb{R}^{F_l} \times \mathbb{B}^{F_b}, \end{aligned}$$

where $\bar{\nu}_k$ is defined in Theorem 4. Then $a^\star$ is a maximizer for state s in Equation (4).

We close this section with some refinements of Algorithm 1 that, while not improving upon its theoretical worst-case complexity, help to speed up the solution of typical problem instances.

Remark 1 (Practical Considerations). *Instead of adding a single most violated constraint in each iteration of Algorithm 1, one can add any constraint (possibly multiple ones) whose violation exceeds a given threshold. Such constraints can be identified from a prematurely terminated solution of the MILP from Theorem 4 or any other heuristic. Moreover, since subsequent instances of the master problem in Algorithm 1 differ only through the inclusion of additional constraints, one can warm-start the LP solver from a previous solution.*

5 Extensions

We now turn to two complementary design choices left open by our framework. Section 5.1 investigates how to construct a compact and informative feature set from a given description of the transition dynamics. Section 5.2 examines how to identify a small yet expressive set of basis functions for our value function approximation.

5.1 Feature Learning

Assume that a decision-maker wishes to solve a fully specified—but intractably large—FMDP $(\mathcal{S}, \mathcal{A}, q, p, r, \gamma)$. Intractability may arise either because no feature map is specified (*i.e.*, the FMDP is given as in Definition 1 rather than Definition 2) or because the number F of features is too large for our cutting plane scheme from Section 4. Our goal in this section is to learn a small set of features that approximates the transition kernel p (and, analogously, the one-step rewards r) as accurately as possible. For clarity, we focus on p in the following; the same ideas apply to r .

Ideally, we would solve the following problem (*cf.* (1)):

$$\begin{aligned} & \underset{F, \phi, \psi}{\text{minimize}} && \sum_{n \in [N]} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \|p_n(\cdot | s, a) - \psi_n(\phi(s, a))\|_2^2 + \lambda \cdot F \\ & \text{subject to} && F \in \mathbb{N}, \quad \phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{B}^F \text{ and } \psi_n : \mathbb{B}^F \rightarrow \Delta(\mathcal{S}_n), n \in [N]. \end{aligned} \tag{5}$$

The decision variables in this problem are the number F of features employed by the approximation, the feature map ϕ , and a set of functions $\psi = \{\psi_n\}_{n \in [N]}$ that map feature realizations to transition probabilities. The Lasso-type penalty $\lambda \cdot F$ with the regularization weight parameter λ trades off approximation accuracy and model size.

Problem (5) is difficult to solve because it optimizes over the high-dimensional functions ϕ and ψ . We therefore consider a variant of the problem that selects the best features from a finite set of pre-specified candidate features $\Phi \subseteq \{\varphi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{B}\}$:

$$\begin{aligned} & \underset{\Gamma, \psi}{\text{minimize}} && \sum_{n \in [N]} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \|p_n(\cdot | s, a) - \psi_n(\phi_\Gamma(s, a))\|_2^2 + \lambda \cdot |\Gamma| \\ & \text{subject to} && \Gamma \subseteq \Phi \text{ and } \psi_n : \mathbb{B}^{|\Gamma|} \rightarrow \Delta(\mathcal{S}_n), n \in [N]. \end{aligned} \tag{5'}$$

Instead of searching over all possible feature vectors ϕ , Problem (5') considers all subsets Γ of

Algorithm 2 Greedy Feature Learning

- 1: **Input:** Set of candidate features Φ and regularization weight $\lambda > 0$.
- 2: Initialize the incumbent feature set to $\Gamma \leftarrow \emptyset$ and the incumbent objective to $\vartheta^* \leftarrow +\infty$.
- 3: **while** $\Gamma \neq \Phi$ **do**
- 4: **for** $\varphi \in \Phi \setminus \Gamma$ **do**
- 5: Compute the optimal value $\vartheta(\varphi)$ of the problem

$$\begin{aligned}
& \underset{\psi}{\text{minimize}} && \sum_{n \in [N]} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \|p_n(\cdot | s, a) - \psi_n(\phi_{\Gamma \cup \{\varphi\}}(s, a))\|_2^2 \\
& \text{subject to} && \psi_n : \mathbb{B}^{|\Gamma|+1} \rightarrow \Delta(\mathcal{S}_n), \quad n \in [N].
\end{aligned} \tag{6}$$

- 6: **end for**
 - 7: **if** $\min_{\varphi} \vartheta(\varphi) + \lambda < \vartheta^*$ **then**
 - 8: Set $\Gamma \leftarrow \Gamma \cup \{\arg \min_{\varphi} \vartheta(\varphi)\}$ and $\vartheta^* \leftarrow \min_{\varphi} \vartheta(\varphi) + \lambda$.
 - 9: **else**
 - 10: **break**
 - 11: **end if**
 - 12: **end while**
 - 13: **Output:** A (greedy) feature selection $\Gamma \subseteq \Phi$.
-

the candidate features Φ , and the corresponding feature vector ϕ_{Γ} is obtained from stacking the features selected by $\Gamma \subseteq \Phi$. As before, Problem (5') also optimizes over all function families ψ that map feature realizations to transition probabilities.

Proposition 4. *Assume that the candidate features Φ admit an exact recovery of the transition kernel p , that is, there exists ψ such that $\psi_n(\phi_{\Phi}(s, a)) = p_n(\cdot | s, a)$ for all $n \in [N]$ and $(s, a) \in \mathcal{S} \times \mathcal{A}$. Then there exists $\lambda_0 > 0$ such that for all regularization weights $\lambda \in (0, \lambda_0)$, every optimal solution to (5') corresponds to a minimum-cardinality feature set that exactly recovers p .*

The conclusion of Proposition 4 extends to the case where Φ does not enable exact recovery: in that case, for sufficiently small λ , the optimal solutions to (5') are minimum-cardinality feature sets Γ that achieve the best attainable approximation of p within Φ .

Problem (5') is combinatorial because it optimizes over subsets $\Gamma \subseteq \Phi$. We solve it approximately with a greedy heuristic that starts from $\Gamma = \emptyset$ (or any other initial subset of Φ), and that iteratively adds features $\varphi \in \Phi \setminus \Gamma$ which yield the largest marginal improvement in the objective function. The procedure is summarized in Algorithm 2.

Observation 1. For fixed Γ and ϑ , Problem (6) admits the closed-form solution

$$\psi_n^*(\phi) = \frac{1}{|\Omega(\phi)|} \cdot \sum_{(s,a) \in \Omega(\phi)} p_n(\cdot | s, a)$$

with $\Omega(\phi) = \phi_{\Gamma \cup \{\varphi\}}^{-1}(\phi) = \{(s, a) \in \mathcal{S} \times \mathcal{A} : \phi_{\Gamma \cup \{\varphi\}}(s, a) = \phi\}, \quad n \in [N].$

Constructing $\Omega(\phi)$ may be expensive when $|\mathcal{S} \times \mathcal{A}|$ is large. However, when the output dimension of $\phi_{\Gamma \cup \{\varphi\}}$ is small (*i.e.*, when $|\Gamma|$ is small), $\psi_n^*(\phi_{\Gamma \cup \{\varphi\}})$ can be approximated well by random sampling. Specifically, we sample a set of state-action pairs $\hat{\Omega} \subseteq \mathcal{S} \times \mathcal{A}$ uniformly at random and use

$$\hat{\psi}_n(\phi) = \frac{1}{|\hat{\Omega}(\phi)|} \sum_{(s,a) \in \hat{\Omega}(\phi)} p_n(\cdot | s, a), \quad \hat{\Omega}(\phi) = \{(s, a) \in \hat{\Omega} : \phi_{\Gamma \cup \{\varphi\}}(s, a) = \phi\},$$

as an unbiased estimator of $\psi_n^*(\phi)$. Next, we are going to show that this estimator concentrates around its mean $\psi_n^*(\phi)$, and therefore a moderate number of samples suffices to provide a good approximation. Since $\|p_n(\cdot | s, a) - \psi_n^*(\phi)\|_2 \leq \sqrt{2}$ for all (s, a) , the matrix Bernstein inequality (Tropp et al., 2015, Theorem 1.6.3) yields

$$\Pr\left(\left\|\hat{\psi}_n(\phi) - \psi_n^*(\phi)\right\|_2^2 \geq \frac{t^2}{|\hat{\Omega}(\phi)|^2}\right) \leq (1 + S_n) \cdot \exp\left(-\frac{t^2/2}{\sqrt{2}(|\hat{\Omega}(\phi)| + t/3)}\right).$$

Moreover, if $\Pr((s, a) \in \Omega(\phi)) \geq p_0 > 0$ for $p_0 = \Theta(2^{-|\Gamma|})$, Hoeffding's inequality ensures that $|\hat{\Omega}(\phi)| \geq p_0|\hat{\Omega}|/2$ with high probability. Setting $t = |\hat{\Omega}(\phi)|/10$, we obtain

$$\Pr\left(\left\|\hat{\psi}_n(\phi) - \psi_n^*(\phi)\right\|_2^2 \geq 0.01\right) \leq (1 + S_n) \cdot \exp\left(-\frac{|\hat{\Omega}(\phi)|}{300}\right).$$

Thus a sample size $|\hat{\Omega}| = 3000 \cdot \Theta(2^{|\Gamma|}) \cdot \log(1 + S_n)$ suffices to guarantee that

$$\Pr\left(\left\|\hat{\psi}_n(\phi) - \psi_n^*(\phi)\right\|_2^2 \geq 0.01\right) \leq 0.01.$$

5.2 Dynamic Basis Approximations

The choice of basis functions ν_k can have a critical impact on the quality of the policy returned by Algorithm 1. In some domains, problem-specific basis functions are known, such as in the computer game Tetris, in queueing networks as well as in inventory control (de Farias and Van Roy, 2003;

Algorithm 3 Dynamic Basis Approximations

```
1: Input: Initial set  $\Gamma$  of basis functions and improvement threshold  $\epsilon \geq 0$ .
2: while True do
3:   Run Algorithm 1 with the basis functions from  $\Gamma$ . Let  $F'$  be the final objective value.
4:   Select a candidate basis function  $\nu^*$  to add to  $\Gamma$ ; break if none found.
5:   Resolve the final master problem from Algorithm 1 using the basis  $\Gamma \cup \{\nu^*\}$ .
   Let  $F^*$  be the final objective value.
6:   if  $F^* \leq F' - \epsilon$  then
7:     Update the basis to  $\Gamma \leftarrow \Gamma \cup \{\nu^*\}$ .
8:   else
9:     break
10:  end if
11: end while
12: Output: Optimized value to-go approximation  $w^*$  from Algorithm 1 for the basis  $\Gamma$ .
```

Powell et al., 2004). However, in other applications such problem-specific knowledge may not be available, and one must resort to more generic constructions. A natural fallback is to include all scope-1, scope-2 or scope-3 basis functions with binary components, which—as our numerical results will show—often performs well, but which may scale poorly in large problems.

To address this challenge, we propose an incremental strategy: start Algorithm 1 with a small initial basis, then iteratively expand the basis by adding promising functions and re-running a warm-started version of Algorithm 1. This approach balances computational tractability with approximation quality. The overall procedure is summarized in Algorithm 3. After the first iteration of the while-loop in Algorithm 3, subsequent runs of Algorithm 1 in Step 3 can be substantially accelerated by warm-starting with the constraint set $\mathcal{C}$ from the previous iteration.

The crucial step in Algorithm 3 is Step 4, where a candidate function ν^* is selected for inclusion in the basis. We now discuss two complementary strategies for this step: one that selects ν^* from a pre-specified candidate set (Section 5.2.1), and one that generates ν^* from scratch (Section 5.2.2).

5.2.1 Dynamic Basis Function Addition

Consider a set of candidate basis functions Ω . In Step 4 of Algorithm 3, we examine each function $\nu \in \Omega \setminus \Gamma$ for possible inclusion in the current basis. In principle, this could be accomplished by re-solving the final master problem of Algorithm 1 and evaluating the change in its optimal objective value F' . In practice, however, this approach is computationally prohibitive. Instead, in analogy

to LP column generation, we approximate this effect by computing the reduced costs of ν :

$$\sum_{s \in \bar{\mathcal{S}}[\nu]} \nu(s) \prod_{n \in \bar{\mathcal{S}}[\nu]} q_n(s_n) - \sum_{(s,a) \in \mathcal{C}} \lambda_{sa}^* \cdot \left(\nu(s) - \gamma \sum_{s' \in \bar{\mathcal{S}}[\nu]} \nu(s') \prod_{n \in \bar{\mathcal{S}}[\nu]} p_n(s'_n | \phi(s, a)) \right).$$

Here, λ_{sa}^* , $(s, a) \in \mathcal{C}$, denote the shadow prices of the constraints in the final master problem of Algorithm 1. The reduced costs provide a conservative estimate of the improvement in objective value if ν were added to the basis Γ : they are computed under the assumption that the current optimal basis remains optimal after augmentation.

We select the candidate $\nu \in \Omega \setminus \Gamma$ with the most negative reduced cost, and we terminate the procedure once either $\Gamma = \Omega$ or all reduced costs are non-negative.

5.2.2 Dynamic Basis Function Generation

Instead of selecting basis functions from a pre-defined set, we can generate them from scratch. To this end, we solve the following problem:

$$\begin{aligned} & \underset{w}{\text{minimize}} && \sum_{k \in [K+1]} w_k \sum_{s \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s) \prod_{n \in \bar{\mathcal{S}}[\nu_k]} q_n(s_n) \\ & \text{subject to} && \sum_{k \in [K+1]} w_k \cdot \nu_k(s) \geq \sum_{j \in [J]} r_j(\phi(s, a)) + \gamma \sum_{k \in [K+1]} w_k \sum_{s' \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s') \prod_{n \in \bar{\mathcal{S}}[\nu_k]} p_n(s'_n | \phi(s, a)) \\ & && \forall (s, a) \in \mathcal{C} \\ & && w \in \mathbb{R}^{K+1} \text{ with } w_{K+1} \equiv 1, \quad \nu_{K+1} : \mathcal{S} \rightarrow \mathbb{R} \text{ with } |\bar{\mathcal{S}}[\nu_{K+1}]| \leq \sigma. \end{aligned} \tag{7}$$

Here, we optimize jointly over the weights $w_1, \dots, w_K$ of the existing basis functions $\Gamma = \{\nu_1, \dots, \nu_K\}$ and the new function ν_{K+1} to be added to the basis. Since the weight w_{K+1} can be absorbed into ν_{K+1} , we fix $w_{K+1} \equiv 1$ to avoid bilinearities. To preserve tractability of Algorithm 1, the block-wise scope of ν_{K+1} is restricted by $|\bar{\mathcal{S}}[\nu_{K+1}]| \leq \sigma$ for some pre-specified constant $\sigma \in \mathbb{N}$.

Problem (7) is computationally challenging: it involves a large number $|\mathcal{S}|$ of decision variables as well as the combinatorial scope constraint $|\bar{\mathcal{S}}[\nu_{K+1}]| \leq \sigma$. We address both difficulties by enumerating all scopes $\bar{\mathcal{S}} \subseteq [N]$ with $|\bar{\mathcal{S}}| \leq \sigma$ and solving, for each, a *sparse basis generation problem*.

In this reduced problem, we fix $\bar{\mathbf{s}}[\nu_{K+1}] = \bar{\mathbf{s}}$ and optimize over the reduced set of decision variables

$$w \in \mathbb{R}^{K+1} \text{ with } w_{K+1} \equiv 1 \quad \text{and} \quad \nu_{K+1} : \left[\bigtimes_{n \in \bar{\mathbf{s}}} \mathcal{S}_n \right] \rightarrow \mathbb{R}.$$

We then select the candidate basis function ν_{K+1}^* corresponding to the block-wise scope $\bar{\mathbf{s}}$ that yields the smallest objective value among all sparse basis generation problems. The procedure terminates once this smallest objective value coincides with F' .

6 Numerical Experiments

We investigate the numerical performance of our proposed methodology on three problems: the classical system administrator problem from the FMDP literature (Schuurmans and Patrascu, 2001; Guestrin et al., 2003), a factored multi-armed bandit problem from the weakly coupled MDP literature, and the machine replacement problem discussed in earlier sections of this paper. Section 6.1 introduces the system administrator problem and compares the performance of our cutting plane scheme from Section 4 with the variable-elimination method of Guestrin et al. (2003) for FMDPs, as well as the sampling approach and the primal-dual approach of Chen et al. (2018) for ADPs. Section 6.2 studies a factored multi-armed bandit problem and compares our FMDP approach with the weakly coupled dynamic programming scheme of Brown and Zhang (2022). Section 6.3 revisits the machine replacement problem introduced earlier and demonstrates how feature learning (*cf.* Section 5.1) allows us to efficiently approximate otherwise intractable transition dynamics. Section 6.4, finally, uses the system administrator problem to illustrate how the dynamic basis approximations from Section 5.2 can yield tighter approximations in shorter runtimes. All algorithms are implemented in C++ using Gurobi 9.5.1 and run on Intel Xeon 2.20GHz cluster nodes with 64 GB dedicated main memory in single-core mode. Our documented sourcecodes, data sets and additional results can be found online (*cf.* Footnote 1).

6.1 Comparison with FMDP and ADP Solution Schemes

Consider a system administrator that maintains a network of N computers. The computers are organized into one of five distinct topologies, as illustrated in Figure 2. In this figure, the computers are represented as nodes, and their connections are depicted as unidirectional or bidirectional arcs.

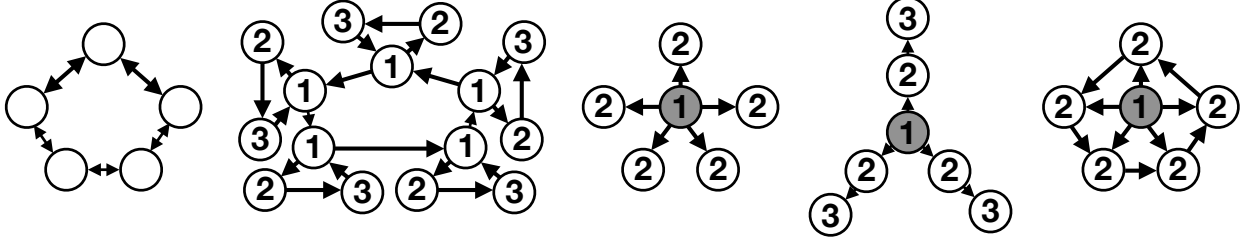


Figure 2. Network topologies for the system administrator problem (from left to right): bidirectional ring, ring of rings, star, 3 legs and ring and star. The first two topologies are decentralized, whereas the others rely on centralized servers (shown as shaded nodes).

In every time period, each computer is in one of three states: inoperative, semi-operational and fully operational. The state of each machine deteriorates stochastically over time, depending on its own current state as well as the state of its predecessors in the topology. To uphold the network's functionality, the system administrator can reboot and thus restore to a fully operational state up to 2 computers in each time period. The administrator earns a per-period reward of 1 unit (2 units) for every computer in a semi-operational (fully operational) state.

The problem can be formulated as an FMDP with the sub-state spaces $\mathcal{S}_n = \{e_1, e_2, e_3\} \subseteq \mathbb{B}^3$, $n \in [N]$, and the action space $\mathcal{A} = \{a \in \mathbb{B}^N : \sum_{n \in [N]} a_n \leq 2\}$. Under a naïve implementation, each set of transition probabilities p_n has a scope between 4 and 10 (accounting for the computer's current state and those of the predecessor nodes, along with the action linked to the computer), whereas each reward r_n has a scope of 3 (accounting for the state of the associated computer). In our experiments we employ scope-1, scope-2 or scope-3 value functions. The scope-1 value functions comprise 3 basis functions for each computer $n \in [N]$: $\nu_{(n,i)} = \mathbf{1}[s_n = e_i]$ for $i \in \{1, 2, 3\}$. The scope-2 value functions comprise 9 basis functions for each computer pair $(n, n \oplus 1)$, where $n \oplus 1 = n + 1$ for $n \in [N - 1]$ as well as $N \oplus 1 = 1$: $\nu_{(n,i),(n \oplus 1,j)} = \mathbf{1}[s_n = e_i \text{ and } s_{n \oplus 1} = e_j]$ for $i, j \in \{1, 2, 3\}$. The scope-3 value functions, finally, comprise 27 basis functions for each computer triplet $(n, n \oplus 1, n \oplus 2)$ that are defined analogously.

We solve our FMDP Problem (3) using Algorithm 1. The master problem is an LP that is solved directly via Gurobi. The subproblem is an MILP, which is initially solved heuristically by identifying violated constraints $(s, a) \in \mathcal{S} \times \mathcal{A}$ via a coordinate-wise descent approach that starts at a randomly selected constraint $(s, a) \in \mathcal{S} \times \mathcal{A}$ and iteratively replaces each sub-state s_n and sub-action a_n , $n \in [N]$, with a new one that maximally increases the constraint violation until a

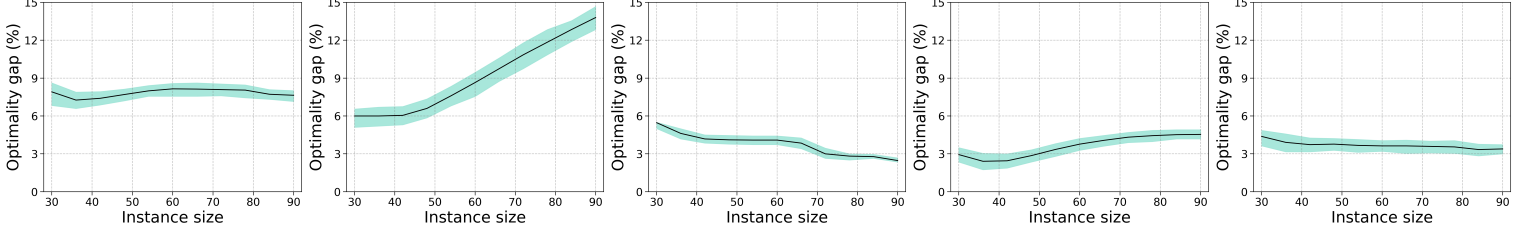


Figure 3. Optimality gaps (in %) for our cutting plane approach on the 5 network topologies of Figure 2 as a function of the instance size (number N of computers).

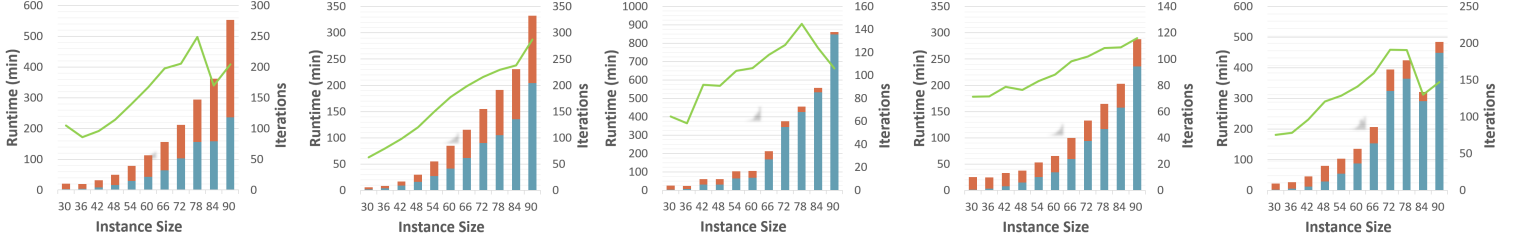


Figure 4. Runtimes (bars) and iteration numbers (bold lines) for our cutting plane approach on the 5 network topologies of Figure 2 as a function of the instance size (number N of computers). The runtime bars comprise the runtimes for the subproblem (upper red bars) and the master problem (lower blue bars).

partial optimum is reached (*cf.* Remark 1 and Korski et al., 2007 as well as §3.3 of Bertsekas, 2009). If the coordinate-wise descent fails to identify a violated constraint, we solve the subproblem as an MILP (*cf.* Theorem 4) with early termination once Gurobi has *(i)* identified a constraint with a sufficiently large violation; *(ii)* concluded that all constraints are satisfied by the current basis function approximation; or *(iii)* exceeded the time limit of $100 + 3N$ seconds.

Figure 3 reports the optimality gaps of our method for the different network topologies of Figure 2 under scope-2 value functions. In the figure, the optimality gaps are computed as $100\% \cdot (\text{UB} - \text{LB})/\text{LB}$, where UB denotes the upper bound on the expected total discounted reward of the unknown optimal policy that is provided by the optimal value of terminal master problem of Algorithm 1 (*cf.* the discussion after Proposition 2), and LB is an approximation of the expected total discounted reward of the policy computed by Algorithm 1 that is determined via Monte Carlo simulation (200 statistically independent runs comprising 200 transitions each). We repeat the experiment for every network topology and every instance size (measured by the number N of computers) 250 times, and we report the median optimality gaps (bold lines) as well as the 10%- and 90%-quantiles (shaded regions). The figure shows that for most network topologies, the

optimality gaps remain well below 10% and do not grow with the size of the instances. An exception is the ring of rings topology, where each computer can connect with as many as 4 other computers. This complexity suggests that scope-2 value functions lack the flexibility to closely approximate the true value function.

Figure 4 presents the runtimes and iteration counts of our method for the different network topologies of Figure 2 under scope-2 value functions. The figure reports the median runtimes across 250 runs, split up into the times spent on solving the subproblem (upper red bars) and the master problem (lower blue bars). The median iteration numbers are reported as bold lines. The figure shows that both the number of iterations and the overall time spent increases with problem size. There is no clear trend concerning the computational bottleneck: for some network topologies, the majority of time is spent on solving the master problem, whereas in some topologies the subproblem consumes an increasing share of the runtime as instances become larger.

To put our results into perspective, we compare the policies and runtimes of our cutting plane approach (‘ours’) with those of the variable-elimination (‘VE’) method by Guestrin et al. (2003), an approximate dynamic programming scheme (‘ADP’), and the primal-dual approach (‘P-D’) of Chen et al. (2018) on the bidirectional ring topology. The ADP scheme replaces the $|\mathcal{S} \times \mathcal{A}|$ Bellman constraints in Problem (3) with a uniformly at random sampled subset of constraints. To this end, we experimented with sampling $\{50, 100, 150, 200, 250, 300\} \cdot K$ constraints, where K is the number of basis functions employed, resulting in master problems with 10^4 – 10^6 constraints. We obtained the best results for $200 \cdot K$ (scope-1) and $250 \cdot K$ (scope-2 and scope-3) constraints, which are reported in the table. For the primal-dual approach, we approximate the primal decision variables v (representing the value to-go function) and the dual decision variables u (representing the state-action occupancy measures) in the LP formulation of the FMDP with

$$v(s) = \sum_{k \in [K]} w_k \cdot \nu_k(s) \quad \text{and} \quad u(s, a) = \sum_{a' \in \mathcal{A}} \sum_{k \in [K]} \eta_{a'k} \cdot \nu_k(s) \cdot \mathbf{1}[a' = a],$$

respectively, where $\nu_k : \mathcal{S} \rightarrow \mathbb{R}$ are the same basis functions as in our approach and where w and η are the new decision variables. We also include two naïve benchmark strategies: a ‘random’ strategy that selects the computers to reboot randomly from those that are inoperative or semi-operational, and a ‘priority’ strategy that prioritizes the rebooting of inoperative computers over that of semi-operational ones. The results are summarized in Table 1. Missing entries correspond

		10	15	20	25	30	35	40
rewards (gaps)	P-D (scope-1)	248.2	348.1	448.4	548.2	647.3	748.0	847.8
	P-D (scope-2)	290.6	390.8	496.6	601.0	705.4	807.7	909.9
	ADP (scope-1)	338.6	446.0	536.9	618.3	710.3	803.3	907.4
	ADP (scope-2)	343.2	465.4	571.3	635.9	736.0	832.9	919.6
	ADP (scope-3)	344.5	473.8	579.4	639.8	733.1	825.6	929.9
	random	336.1	444.9	528.6	620.8	716.6	816.0	914.2
	priority	338.1	449.8	536.8	633.8	734.0	835.1	938.1
	VE (scope-1)	342.9 (9.2%)	454.9 (23.2%)	534.9 (33.6%)	633.7 (35.9%)	737.0 (36.4%)	838.8 (36.9%)	941.1 (37.1%)
	VE (scope-2)	344.5 (8.7%)	482.2 (7.7%)	—	—	—	—	—
	ours (scope-1)	340.4 (10.0%)	454.0 (23.4%)	533.2 (35.1%)	619.4 (37.6%)	726.1 (33.5%)	821.0 (33.4%)	929.3 (31.9%)
	ours (scope-2)	343.1 (9.1%)	481.6 (8.2%)	587.2 (7.3%)	682.2 (8.5%)	778.2 (8.2%)	877.3 (8.6%)	975.4 (8.5%)
	ours (scope-3)	344.5 (6.1%)	482.5 (5.2%)	592.8 (2.8%)	690.0 (3.3%)	789.0 (3.1%)	888.0 (3.4%)	988.3 (3.1%)
runtimes (s)	P-D (scope-1)	0.2	1.1	3.3	7.8	15.7	28.7	48.8
	P-D (scope-2)	2.1	10.9	36.8	89.4	184.7	341.3	579.7
	ADP (scope-1)	0.7	1.4	2.5	3.9	5.5	7.1	14.7
	ADP (scope-2)	5.3	12.4	21.6	34.5	56.9	71.4	142.6
	ADP (scope-3)	45.0	103.9	200.8	342.1	593.3	738.1	1,463.8
	VE (scope-1)	4.1	59.2	235.2	1,149.3	2,404.0	16,134.2	56,914.1
	VE (scope-2)	899.0	19,627.1	—	—	—	—	—
	ours (scope-1)	86.8	146.3	161.9	418.7	243.3	400.3	686.2
	ours (scope-2)	131.2	259.9	170.9	622.8	404.8	798.3	1,437.9
	ours (scope-3)	140.8	469.3	901.6	1,571.5	2,134.6	4,063.5	7,323.8

Table 1. Expected total discounted reward, optimality gaps and runtimes for the variable-elimination method (‘VE’), the approximate dynamic programming scheme (‘ADP’), the primal-dual approach (‘P-D’), two naïve benchmark strategies (‘random’ and ‘priority’) as well as our cutting plane approach (‘ours’) on the bidirectional ring topology. Absent results (“—”) indicate that the corresponding experiments have run out of memory.

to experiments that ran out of memory. The table shows that our cutting plane approach results in similar optimality gaps as the variable-elimination method at substantially lower runtime and memory requirements. The table also shows that the two naïve benchmark strategies—apart from being unable to provide optimality gaps—may underperform by up to 10% relative to our scope-3

	12	24	36	48	60
P-D (scope-1)	320.8	585.8	876.1	1,031.1	1,283.8
P-D (scope-2)	374.7	667.3	890.6	1,134.2	1,392.7
ADP (scope-1)	413.3	770.8	881.9	1,089.1	1,314.3
ADP (scope-2)	421.1	688.9	862.8	1,065.7	1,340.4
ADP (scope-3)	417.9	767.4	829.6	1,055.0	1,303.3
random	413.9	694.7	917.9	1,149.7	1,386.2
priority	416.2	714.8	958.9	1,208.3	1,454.6
level	428.5	787.2	1,071.1	1,297.9	1,524.8
ours (scope-1)	429.4 (5.8%)	801.6 (10.1%)	1,122.1 (11.6%)	1,401.4 (13.2%)	1,670.8 (15.2%)
ours (scope-2)	430.9 (4.4%)	804.8 (6.6%)	1,131.12 (7.1%)	1,417.1 (7.9%)	1,679.5 (8.7%)
ours (scope-3)	431.5 (3.2%)	807.3 (4.3%)	1,142.3 (4.7%)	1,425.7 (5.2%)	1,688.3 (5.8%)

Table 2. Expected total discounted reward for the three benchmark strategies as well as our cutting plane approach on the ring of rings topology. In contrast to the benchmark strategies, our approach additionally provides optimality gaps, which are listed as well.

strategy. The primal-dual approach and the dynamic programming scheme outperform our cutting plane approach in terms of runtimes, but they result in substantially inferior policies. In fact, even the greedy ‘priority’ strategy outperforms both the primal-dual approach and the dynamic programming scheme for large problem sizes. Our findings are further emphasized in Table 2, which compares the benchmark strategies with our cutting plane approach for larger instances sizes on the ring of rings topology. Here, even the more sophisticated ‘priority’ strategy is outperformed by our scope-1 value functions, and our scope-3 value functions generate up to 19% more expected total discounted reward. As before, both the primal-dual approach and the dynamic programming scheme are outperformed by the ‘priority’ strategy on larger problem instances. Analogous comparisons for the star, 3-leg and ring-and-star topologies are provided in the GitHub repository accompanying this work. The results are qualitatively consistent with those reported here: across all topologies, both the primal-dual approach and the approximate dynamic programming scheme produce substantially inferior policies compared to our cutting plane approach, and they are again outperformed by the naïve benchmark strategies on larger problem instances. The ring of rings topology allows us to distinguish between nodes of different levels as shown in Figure 2, ranging from 1 (highest) to 3 (lowest). We therefore also include a ‘level’ heuristic which breaks ties among

		ring of rings			star		3 legs			ring and star	
		1	2	3	1	2	1	2	3	1	2
inoperative	prob	0.4	1.9	2.8	0.0	0.9	0.0	1.9	6.8	0.0	2.4
	reboot	94.8	26.5	6.0	—	88.2	—	49.2	17.4	—	36.9
semi-operational	prob	10.6	21.2	39.2	5.0	17.1	5.1	20.7	44.1	5.0	37.6
	reboot	63.1	17.1	2.2	100.0	18.8	100.0	21.8	3.6	100.0	8.7

Table 3. Probability of a computer being inoperative or semi-operational (‘prob’; in %) and the corresponding conditional reboot probability (‘reboot’) for computers of different levels under different network topologies.

inoperative and semi-operational computers based on the priorities of the involved computers. We see that this heuristic, too, is outperformed by all of our policies.

So far, our discussion has focused on a quantitative analysis of the suboptimality incurred and the runtimes as well as the memory consumed by different algorithms. Table 3 offers some insights into the qualitative structure of the determined policies. In particular, the table reveals the probabilities with which different computers are in an inoperative or semi-operational state in any given time period, as well as the conditional probabilities with which those computers are rebooted in such time periods, under the policies determined by our cutting plane approach. To this end, the table distinguishes between nodes of different levels as shown in Figure 2, ranging from 1 (highest) to 3 (lowest). The table omits the bidirectional ring topology as it does not contain nodes of different levels. In line with our intuition, the policies generated by our cutting plane approach reboot more promptly computers at lower levels, which in turn implies that those computers have a higher probability of being in a fully operational state.

6.2 Comparison with Weakly Coupled MDP Solution Schemes

Our second experiment follows Brown and Zhang (2022) and considers a multi-armed bandit problem with M arms over an infinite time horizon. Each arm $m \in [M]$ transitions between S_m different states. There are two actions associated with each arm $m \in [M]$, $a_m = 0$ (‘do nothing’) and $a_m = 1$ (‘pull arm’), and we can pull up to B arms in each time period. An arm remains in its current state under the ‘do nothing’ action and does not generate any rewards, whereas it stochastically transitions into a new state and generates a random reward under the ‘pull arm’ action. The transition probabilities and rewards of each arm depend on the arm’s current state as well as a shared signal z that evolves exogenously according to a Markov chain with S_{M+1} states.

The problem can be naturally formulated as an FMDP with sub-state spaces $\mathcal{S}_m = \{e_1, \dots, e_{S_m}\}$, $m \in [M+1]$, where the first M sub-states record the states of the arms and the last sub-state keeps track of the shared signal, respectively, and the action space $\mathcal{A} = \{a \in \mathbb{B}^M : \sum_{m \in [M]} a_m \leq B\}$. In our experiments, we consider $M \in \{20, 50, 80\}$ arms with a homogeneous number $S_m \in \{2, 3, 4\}$ of states each. The shared signal, on the other hand, transitions between $S_{M+1} = 5$ states throughout our experiments. The initial distributions and transition probabilities of each arm m and the shared signal z are selected uniformly at random from the corresponding probability simplices, and the rewards of the ‘pull arm’ actions are selected uniformly at random from the interval $[0, 1]$. We fix $B = \beta \cdot M$ where $\beta \in \{5\%, 10\%, 15\%, 20\%\}$ denotes the percentage of arms that can be pulled in each time period. Since a simple greedy policy turns out to perform well across this class of problems, we random select $\eta \cdot M$ arms, $\eta \in \{0\%, 10\%, 20\%\}$, as ‘special arms’ whose rewards are zero under any state other than state 0, and whose rewards in state 0 under the ‘pull arm’ action are inflated by a factor of S_m to maintain a long term profitability that is comparable to that of the other arms. Throughout our experiments, we use the value functions $\nu_{mij}(s) = \mathbf{1}[s_m = e_i \text{ and } s_{M+1} = e_j]$ for $m \in [M]$, $i \in [S_m]$ and $j \in [S_{M+1}]$.

Similar to our earlier experiments in Section 6.1, we solve our FMDP Problem (3) using Algorithm 1. The factored multi-armed bandit problem also allows for a representation as a weakly coupled MDP, and we thus compare our FMDP approach with the dynamic fluid policy of Brown and Zhang (2022). Since this policy is restricted to finite horizon problems, we fix its time horizon to 150 periods and discount the rewards geometrically. We also consider a greedy policy that pulls the B arms with largest one-step rewards in any particular time period. Table 4 compares the out-of-sample rewards generated by these three policies in terms of relative gap to the (typically unachievable) upper reward bound provided by the fluid relaxation of Brown and Zhang (2022). The table shows that our FMDP approach dominates both benchmark approaches over the considered class of instances. That said, we expect the dynamic fluid policy to outperform our FMDP strategy for large problem sizes since it enjoys asymptotic optimality guarantees.

6.3 Feature Learning

We revisit the machine replacement problem from Examples 1–5. Assume that $N = 10$ machines are operated in sequence. The condition of each machine $n \in [N]$ is represented by its state

			20 arms				50 arms				80 arms			
			5%	10%	15%	20%	5%	10%	15%	20%	5%	10%	15%	20%
2 states	0% S-arms	B&Z	7.56%	6.52%	5.88%	5.62%	6.98%	5.76%	5.29%	4.90%	6.28%	5.48%	5.06%	4.66%
		GRD	1.60%	1.94%	2.09%	2.35%	0.95%	1.31%	1.61%	1.83%	0.91%	1.25%	1.50%	1.65%
		FMDP	1.13%	1.15%	1.33%	1.50%	0.97%	1.89%	2.05%	2.17%	1.89%	2.40%	2.45%	2.47%
	10% S-arms	B&Z	8.21%	6.63%	6.21%	5.81%	7.07%	5.91%	5.52%	5.03%	6.34%	5.77%	5.18%	4.82%
		GRD	6.51%	5.37%	5.06%	5.33%	5.99%	4.75%	4.55%	4.73%	5.23%	4.88%	4.49%	4.84%
		FMDP	1.31%	1.18%	1.33%	1.48%	1.15%	2.06%	2.26%	2.30%	2.28%	2.69%	2.70%	2.63%
	20% S-arms	B&Z	8.22%	6.71%	6.14%	5.82%	7.39%	6.05%	5.78%	5.37%	6.51%	5.76%	5.19%	5.04%
		GRD	8.23%	8.17%	8.40%	8.83%	8.70%	8.05%	7.67%	7.90%	8.83%	8.12%	7.44%	7.81%
		FMDP	1.65%	1.20%	1.22%	1.53%	1.73%	2.30%	2.34%	2.38%	2.90%	3.04%	3.00%	3.00%
3 states	0% S-arms	B&Z	10.62%	9.22%	8.46%	7.89%	9.40%	8.10%	7.60%	6.77%	8.49%	7.56%	6.90%	6.55%
		GRD	2.25%	2.74%	3.12%	3.44%	1.42%	2.04%	2.40%	2.73%	1.38%	1.89%	2.22%	2.59%
		FMDP	1.73%	1.82%	1.97%	2.02%	1.37%	2.23%	2.41%	2.48%	2.22%	2.72%	2.83%	3.06%
	10% S-arms	B&Z	11.51%	9.62%	8.73%	8.02%	10.65%	8.55%	7.81%	7.18%	9.17%	8.02%	7.37%	6.83%
		GRD	7.13%	7.73%	7.03%	7.39%	9.66%	7.68%	7.04%	6.77%	9.33%	6.77%	6.76%	6.77%
		FMDP	1.84%	1.68%	1.79%	1.89%	1.68%	2.17%	2.38%	2.52%	2.36%	2.72%	3.02%	3.11%
	20% S-arms	B&Z	12.10%	10.13%	9.16%	8.67%	10.85%	9.02%	8.25%	7.49%	9.42%	8.32%	7.72%	7.10%
		GRD	13.06%	12.01%	11.69%	11.55%	14.12%	11.36%	10.71%	10.61%	13.91%	11.67%	11.19%	11.20%
		FMDP	2.74%	1.76%	1.71%	1.90%	2.44%	2.20%	2.38%	2.53%	2.92%	2.87%	3.00%	3.08%
4 states	0% S-arms	B&Z	12.29%	10.66%	9.62%	8.72%	10.85%	9.12%	8.58%	7.88%	9.59%	8.62%	7.97%	7.51%
		GRD	2.71%	3.29%	3.66%	3.94%	1.68%	2.46%	2.87%	3.32%	1.60%	2.25%	2.78%	3.12%
		FMDP	2.24%	2.21%	2.28%	2.29%	1.76%	2.37%	2.57%	2.70%	2.45%	2.92%	3.05%	3.24%
	10% S-arms	B&Z	13.21%	11.57%	10.27%	9.73%	11.86%	9.88%	9.18%	8.28%	10.61%	9.23%	8.47%	7.91%
		GRD	10.94%	9.70%	9.46%	8.96%	12.71%	8.83%	8.54%	8.02%	11.66%	8.13%	7.83%	8.13%
		FMDP	2.66%	2.13%	2.08%	2.21%	2.41%	2.33%	2.50%	2.62%	2.58%	2.80%	3.02%	3.15%
	20% S-arms	B&Z	15.06%	11.90%	10.71%	9.97%	12.77%	9.86%	9.48%	8.82%	11.16%	9.48%	8.98%	8.40%
		GRD	15.58%	14.14%	12.19%	13.22%	17.53%	13.79%	12.79%	12.89%	17.32%	13.85%	12.31%	12.65%
		FMDP	3.65%	2.28%	2.14%	2.12%	3.23%	2.29%	2.48%	2.59%	3.24%	2.85%	3.09%	3.21%

Table 4. Optimality gaps of our FMDP approach (‘FMDP’), the dynamic fluid policy (‘B&Z’) and the greedy policy (‘GRD’) across different sub-state space dimensions S_m and percentages η of special arms (rows) as well as numbers of arms M and percentages β of arms to be pulled (columns). All gaps are averages over 100 random problem instances. The best performance for each instance class is highlighted in bold print.

$s_n \in \mathcal{S}_n = \{e_i \in \mathbb{B}^5 : i \in [5]\} \subseteq \mathbb{B}^5$, where e_1 and e_5 represent malfunction and optimal performance, respectively. We define the action space as $\mathcal{A} = \{e_i \in \mathbb{B}^N : i \in [N]\} \subseteq \mathbb{B}^N$, with the interpretation that $a_n = 1$ if and only if machine n is being repaired. Thus, exactly one machine must be repaired in every period.

In practical applications, the deterioration dynamics of the system may be (approximately) known but complicated; for example, they may be described by systems of nonlinear equations or characterized by simulation models. In such cases, one may aim to approximate the true system with a feature-based FMDP that uses only a limited number of features. For concreteness, we

assume that the deterioration dynamics are characterized by the transition probabilities

$$p(s'_{ni} = 1 \mid s, a) = \begin{cases} \mathbf{1}[i = 5] & \text{if } a_n = 1, \\ 0.6^{\delta_n(s)} & \text{if } a_n = 0 \text{ and } s_{ni} = 1, \\ 1 - 0.6^{\delta_n(s)} & \text{if } a_n = 0 \text{ and } s_{n,i+1} = 1, \\ 0 & \text{otherwise} \end{cases}$$

for $n \in [N]$ and $i \in [5]$, where we set $s_{n,6} \equiv 0$ to economize on notation. That is, machine n transitions into the optimal performance state 5 if it is repaired; otherwise, it either remains in its current state or deteriorates into the next lower state with probabilities $0.6^{\delta_n(s)}$ and $1 - 0.6^{\delta_n(s)}$, respectively. The function $\delta_n(s)$ expresses the likelihood of deterioration of machine n :

$$\delta_n(s) = 1 + \sum_{l=0}^{L(n)} \left(1 - \frac{l}{L(n) + 1} \right) \cdot \rho(s_{n-l})$$

Here, L denotes the *maximum* number of predecessors that can influence any machine's dynamics, $L(n) = \min\{L, n - 1\}$ represents the *actual* number of predecessors influencing machine n , and $\rho(s_l) = \mathbf{1}[s_l \in \{e_1, e_2, e_3\}]$ indicates whether the l -th machine is in a poor state. The function $\delta_n(s)$ therefore models a situation where up to L predecessors can affect the dynamics of machine n , with closer predecessors having greater influence than those further upstream.

Assume that the decision maker wishes to approximate these dynamics through a subset of the 385 candidate features $\Omega = \{\phi_{ni}^1\}_{n,i} \cup \{\phi_n^2\}_n \cup \{\phi_{nk}^3\}_{n,k} \cup \{\phi_{nkl}^4\}_{n,k,l}$:

- (i) 50 features $\phi_{ni}^1(s, a) = \mathbf{1}[s_n = e_i]$, $n \in [10]$ and $i \in [5]$, that indicate the state i of each machine n ;
- (ii) 10 features $\phi_n^2(s, a) = \mathbf{1}[a_n = 1]$, $n \in [10]$, that indicate whether machine n is repaired;
- (iii) 50 features $\phi_{nk}^3(s, a) = \mathbf{1}[s_n = e_i \text{ for some } i \in [k]]$, $n \in [10]$ and $k \in [5]$, that indicate whether machine n is in state k or worse; and
- (iv) 275 features $\phi_{nkl}^4(s, a) = \mathbf{1}[\text{at least } l \text{ out of } i \in [n] \text{ satisfy } s_i = e_j \text{ for some } j \in [k]]$, $n \in [10]$, $k \in [5]$ and $l \in [n]$, that indicate whether at least l of machine n 's predecessors (including n) are in state k or worse.

# of features	1	2	3	4	5	6	7	8	9	10
machine 1	21.8%	10.6%	5.1%	0						
machine 2	27.7%	16.1%	6.6%	2.1%	0.4%	0				
machine 3	27.9%	16.6%	6.6%	2.0%	0.7%	0.1%	0			
machine 4	29.5%	17.3%	7.0%	1.8%	0.8%	0.3%	0.05 %	0		
machine 5	30.7%	18.4%	7.4%	1.6%	0.9%	0.4%	0.2 %	0.08%	0	
machine 6	32.4%	19.8%	8.2%	1.4%	0.8%	0.4%	0.2 %	0.1%	0.05%	0

Table 5. Average estimation loss by machine as a function of the number of machine-specific features. For machine n , the average estimation loss is defined as the average squared ℓ_2 -error in approximating $p_n(\cdot | s, a)$ over $(s, a) \in \mathcal{S} \times \mathcal{A}$. Losses are estimated using the sampling procedure described at the end of Section 5.1.

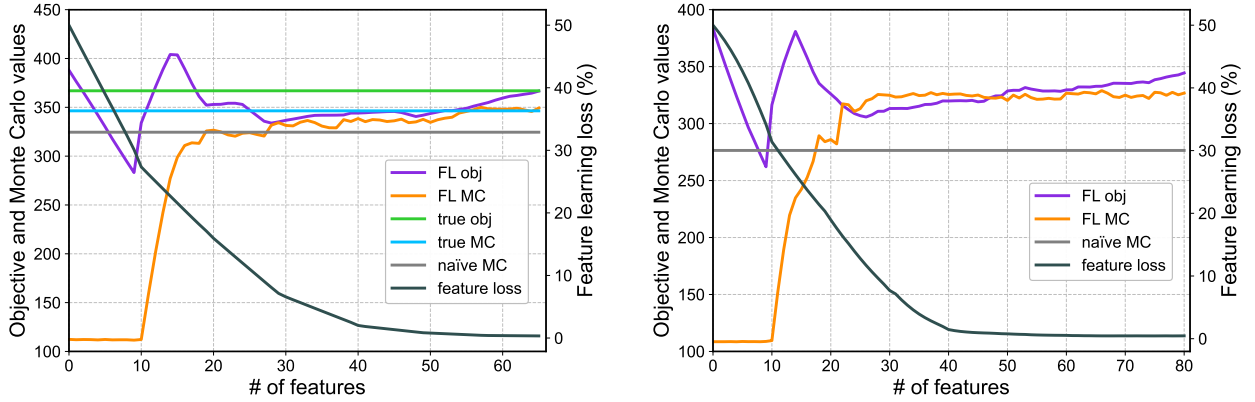


Figure 5. Performance comparison for $L = 2$ (left) and $L = 9$ (right) predecessors impacting the dynamics of a machine.

Table 5 shows that, for each machine considered in isolation, a small number of features suffices to approximate its transition dynamics accurately. By contrast, our feature learning algorithm selects a subset of features that best approximates the transition dynamics of all machines jointly.

Figure 5 compares the performance of three policies: the one obtained with our feature learning approach (‘FL’), the one obtained by solving the FMDP with the true dynamics (‘true’), and a naïve benchmark policy that always repairs the machine in the worst state (‘naïve’). Performance is measured in two ways. First, we report the objective value of the final master problem of Algorithm 1 (‘obj’). This value provides an upper bound on the performance of the unknown optimal policy. The bound is exact when the true dynamics are used, but it is only approximate for FL, since FL relies on estimated transition probabilities in the master problem. Second, we show results from a Monte Carlo simulation of the resulting policies (‘MC’), which provides a lower bound on the expected total discounted reward of the unknown optimal policy. The figure also plots

the associated average estimation losses across all machines (‘feature learning loss’). All quantities are reported as functions of the number of features introduced by the feature learning approach.

The left panel of Figure 5 illustrates the case where up to $L = 2$ predecessors influence a machine’s dynamics. Here, the FMDP with the true dynamics is still tractable, and the results show that a moderate number of features is enough to approximate the true optimal policy closely. The right panel considers the case where up to $L = 9$ predecessors may influence the dynamics. In this setting, solving the FMDP with the true dynamics is no longer feasible, so it is omitted. Even though the true optimal policy is unknown in this case, the results suggest that the feature learning approach converges towards an approximate optimum. Finally, note that the lower and upper bounds of the feature learning approach may cross, because the master problems rely on estimated probabilities, whereas the Monte Carlo simulations are based on the true transition probabilities.

6.4 Dynamic Basis Approximation

We revisit the system administrator problem from Section 6.1, focusing on the bidirectional ring topology with 30 machines and scope-2 value functions. Recall that these value functions were originally constructed from 9 basis functions for each adjacent computer pair $(n, n \oplus 1)$. This construction yields 270 basis functions in total. We extend this set by also including pairs $(n, n \oplus 2)$, resulting in 540 basis functions, and further by including $(n, n \oplus 3)$, resulting in 810 basis functions. Because of the ring structure, a machine is most influenced by its nearest neighbors; we therefore restrict attention to these localized basis functions rather than all possible scope-2 functions.

Figure 6 compares the performance of dynamic basis addition (‘ADD’; *cf.* Section 5.2.1) and dynamic basis generation (‘GEN’; *cf.* Section 5.2.2) against the standard cutting plane method with fixed sets of 540 or 810 basis functions (*cf.* Algorithm 1). As in the previous section, ‘obj’ reports the objective value of the final master problem of Algorithm 1, whereas ‘MC’ shows results from a Monte Carlo simulation of the resulting policies. Both dynamic basis approximations match the upper and lower bounds of the fixed basis with 810 functions after substantially less than 810 iterations, thereby achieving notable runtime savings.

Figure 7 further illustrates the complementarity of dynamic basis addition and generation. Although dynamic basis generation requires more time per iteration, it converges in significantly fewer iterations and thus leads to a comparable overall efficiency.

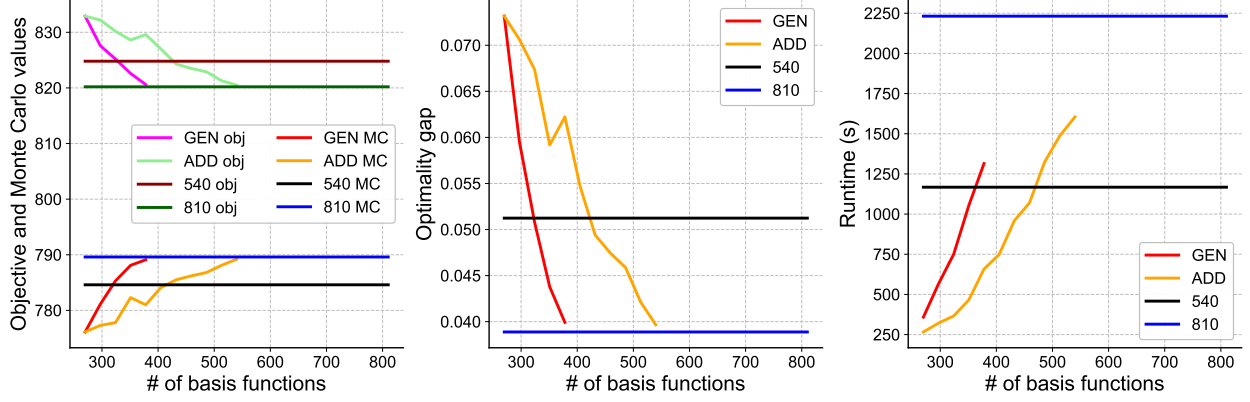


Figure 6. Performance of dynamic basis approximations on the 30-machine ring topology. Shown are upper and lower bounds (left), optimality gaps (middle) and runtimes (right) as functions of the number of basis functions introduced.

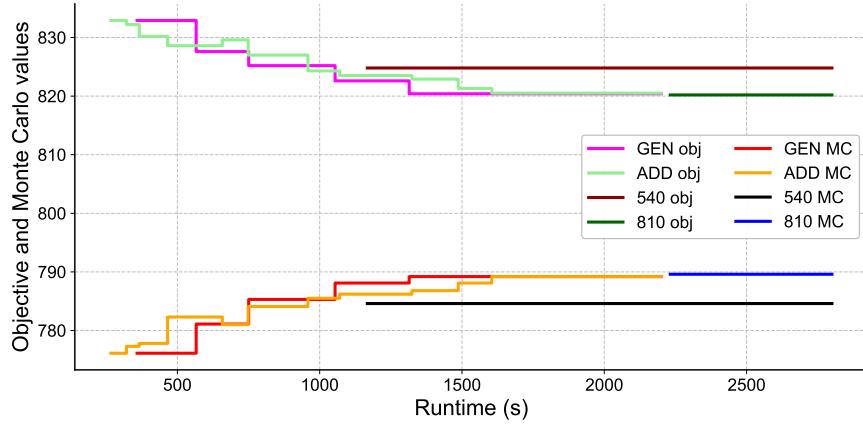


Figure 7. Upper and lower bounds over runtime for our two dynamic basis approximations, as well as our two fixed-basis benchmarks.

Acknowledgments

Huikang Liu gratefully acknowledges funding from NSFC grants 12301403, 72192830, and 72192832. Wolfram Wiesemann's research is funded by EPSRC grant EP/W003317/1. Man-Chung Yue is supported in part by Hong Kong Research Grants Council under the GRF project 17309423.

Biographies

Huikang Liu is an Associate Professor of Antai College of Economics and Management at Shanghai Jiao Tong University. His research interests focus on continuous optimization, with

applications to large-scale computation and mechanism design.

Wolfram Wiesemann is Professor of Analytics & Operations at Imperial College Business School. His research interests evolve around decision-making under uncertainty, with applications to supply chain management, healthcare and energy. Wolfram currently serves as editor in-chief of Operations Research Letters as well as a department co-editor for Management Science.

Man-Chung Yue is an Associate Professor in the Department of Applied Mathematics at The Hong Kong Polytechnic University. His research focuses on continuous optimization and its interplay with machine learning, operations research, data science, and signal processing.

References

- D. Adelman and A. J. Mersereau. Relaxations of weakly coupled stochastic dynamic programs. *Operations Research*, 56(3):712–727, 2008.
- A. Angelidakis and G. Chalkiadakis. Factored MDPs for optimal prosumer decision-making. In *Proceedings of the 14th International Conference on Autonomous Agents and Multiagent Systems*, pages 503–511, 2014.
- A. Angelidakis and G. Chalkiadakis. Factored MDPs optimal prosumer decision-making in continuous state spaces. In *Proceedings of the 13th European Conference on Multi-Agent Systems*, pages 91–107, 2015.
- R. Bellman. On the theory of dynamic programming. *Proceedings of the National Academy of Sciences of the United States of America*, 38(8):716–719, 1952.
- U. Bertele and F. Brioschi. *Nonserial Dynamic Programming*. Academic Press, 1972.
- D. P. Bertsekas. *Dynamic programming and optimal control, Volume I*. Athena Scientific, 1995.
- D. P. Bertsekas. *Convex Optimization Theory*. Athena Scientific, 2009.
- D. P. Bertsekas and J. N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.
- D. Bertsimas and V. V. Mišić. Decomposable Markov decision processes: A fluid optimization approach. *Operations Research*, 64(6):1537–1555, 2016.
- C. Boutilier, R. Dearden, and M. Goldszmidt. Exploiting structure in policy construction. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 1104–1113, 1995.
- C. Boutilier, R. Dearden, and M. Goldszmidt. Stochastic dynamic programming with factored representations. *Artificial Intelligence*, 121(1–2):49–107, 2000.
- D. B. Brown and J. Zhang. Dynamic programs with shared resources and signals: Dynamic fluid policies and asymptotic optimality. *Operations Research*, 70(5):2597–3033, 2022.
- R. E. Bryant. On the complexity of VLSI implementations and graph representations of Boolean functions with application to integer multiplication. *IEEE Transactions on Computers*, 40(2):205–213, 1991.
- Y. Chen, L. Li, and M. Wang. Scalable bilinear π learning using state and action features. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 834–843, 2018.

- D. P. de Farias and B. Van Roy. The linear programming approach to approximate dynamic programming. *Operations Research*, 51(6):850–865, 2003.
- T. Degris, O. Sigaud, and P.-H. Wuillemin. Learning the structure of factored Markov decision processes in reinforcement learning problems. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 257–264, 2006.
- K. V. Delgado, S. Sanner, and L. N. de Barros. Efficient solutions to factored MDPs with imprecise transition probabilities. *Artificial Intelligence*, 175(9–10):1498–1527, 2011.
- D. Dolgov and E. Durfee. Resource allocation among agents with preferences induced by factored MDPs. In *Proceedings of the 5th International Joint Conference on Autonomous Agents and Multiagent Systems*, pages 297–304, 2006.
- M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co, 1979.
- C. E. Guestrin, D. Koller, and R. Parr. Max-norm projections for factored MDPs. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence*, pages 673–680, 2001a.
- C. E. Guestrin, D. Koller, and R. Parr. Multiagent planning with factored MDPs. In *Advances in Neural Information Processing Systems*, volume 14, pages 1523–1530, 2001b.
- C. E. Guestrin, R. E. Patrascu, and D. E. Schuurmans. Exploiting structure in policy construction. In *Proceedings of the 19th International Conference on Machine Learning*, pages 235–242, 2002a.
- C. E. Guestrin, S. Venkataraman, and D. Koller. Context specific multiagent coordination and planning with factored MDPs. In *Proceedings of the 18th National Conference on Artificial Intelligence*, pages 253–259, 2002b.
- C. E. Guestrin, D. Koller, R. Parr, and S. Venkataraman. Efficient solution algorithms for factored MDPs. *Journal of Artificial Intelligence Research*, 19(1):399–468, 2003.
- J. T. Hawkins. *A Lagrangian decomposition approach to weakly coupled dynamic optimization problems and its applications*. PhD thesis, Massachusetts Institute of Technology, 2003.
- J. Hoey, R. St-Aubin, A. Hu, and C. Boutilier. SPUDD: Stochastic planning using decision diagrams. In *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence*, pages 279–288, 1999.
- L. Huang, J. Chen, and Q. Zhu. A factored MDP approach to optimal mechanism design for resilient large-scale interdependent critical infrastructures. In *Proceedings of the 2017 Workshop on Modeling and Simulation of Cyber-Physical Energy Systems*, pages 1–6, 2017.
- J. Korsi, F. Pfeuffer, and K. Klamroth. Biconvex sets and optimization with biconvex functions: A survey and extensions. *Mathematical Methods of Operations Research*, 66(3):373–407, 2007.
- M. L. Littman. Probabilistic propositional planning: Representations and complexity. *Proceedings of the 14th AAAI Conference*, pages 748–754, 1997.
- I. Osband and B. Van Roy. Near-optimal reinforcement learning in factored MDPs. In *Advances in Neural Information Processing Systems*, volume 27, pages 1–9, 2014.
- W. B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Wiley, 2nd edition, 2011.
- W. B. Powell, A. Ruszczyński, and H. Topaloglu. Learning algorithms for separable approximations of discrete stochastic optimization problems. *Mathematics of Operations Research*, 29(4):814–836, 2004.

- M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.
- A. Reyes, P. H. Ibargüengoytia, and L. E. Sucar. Power plant operator assistant: An industrial application of factored MDPs. In *Proceedings of the 3rd Mexican International Conference on Artificial Intelligence*, pages 565–573, 2004.
- A. Reyes, M. T. J. Spaan, and L. E. Sucar. An intelligent assistant for power plants based on factored MDPs. In *Proceedings of the 15th International Conference on Intelligent System Applications to Power Systems*, pages 1–6, 2009.
- B. Sallans and G. E. Hinton. Reinforcement learning with factored states and actions. *Journal of Machine Learning Research*, 5(Aug):1063–1088, 2004.
- D. E. Schuurmans and R. E. Patrascu. Direct value-approximation for factored MDPs. In *Advances in Neural Information Processing Systems*, volume 14, pages 1579–1586, 2001.
- P. Schweitzer and A. Seidmann. Generalized polynomial approximations in Markovian decision processes. *Journal of Mathematical Analysis and Applications*, 110(2):568–582, 1985.
- H. A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.
- L. J. Stockmeyer and A. K. Chandra. Provably difficult combinatorial games. *SIAM Journal on Computing*, 8(2):151–174, 1979.
- M. Tavakol and U. Brefeld. Factored MDPs for detecting topics of user sessions. In *Proceedings of the 8th ACM Conference on Recommender Systems*, pages 33–40, 2014.
- Joel A Tropp et al. An introduction to matrix concentration inequalities. *Foundations and Trends® in Machine Learning*, 8(1-2):1–230, 2015.
- J. D. Williams, P. Poupart, and S. Young. Factored partially observable Markov decision processes for dialogue management. In *Proceedings of the 4th Workshop on Knowledge and Reasoning in Practical Dialog Systems*, pages 76–90, 2005.
- N. Zhang and D. Poole. On the role of context-specific independence in probabilistic reasoning. In *Proceedings of the 16th International Joint Conference on Artificial Intelligence*, pages 1288–1293, 1999.

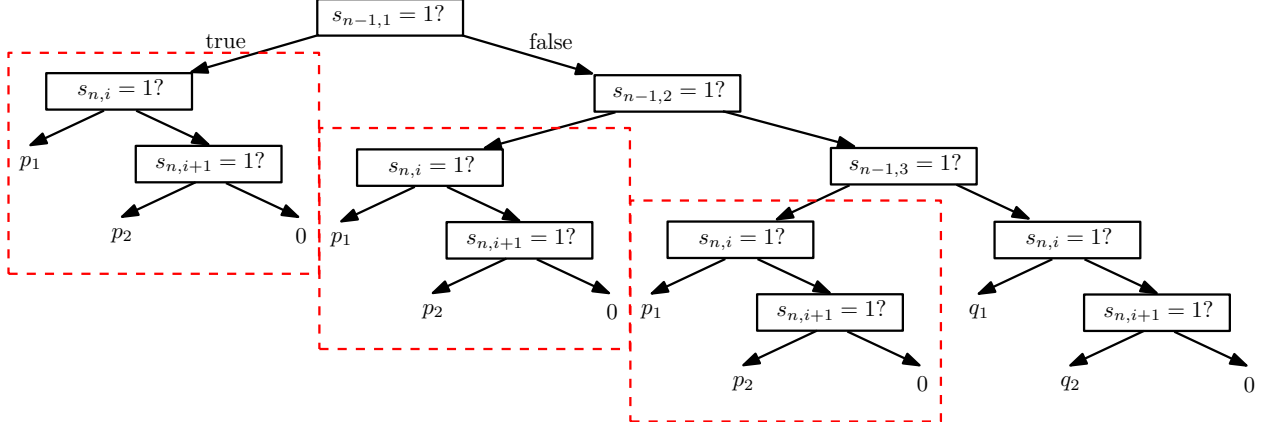
E-Companion A: Context-Specific Independence

A *decision tree* represents a function $f : \mathcal{S} \rightarrow \mathbb{R}$ as a tree that branches upon the values of some of the components of $s \in \mathcal{S}$ and that contains the function values $f(s)$ as leaf nodes. We can exploit context-specific independence in the transition probabilities $p_n(s'_n | s, a)$ by constructing a separate tree for each sub-state $n \in [N]$, each possible value $s'_n \in \mathcal{S}_n$ of that sub-state and each action $a \in \mathcal{A}$. Likewise, we can exploit context-specific independence in the rewards $r(s, a)$ by constructing a separate tree for each reward component r_j , $j \in [J]$, and each action $a \in \mathcal{A}$. The size of a decision tree's description scales with its number of (internal and leaf) nodes.

An *algebraic decision diagram* (ADD) $f : \mathbb{B}^I \rightarrow \mathbb{R}$ of binary vectors $x = (x_1, \dots, x_I)$ is defined recursively as follows: An empty function $f : \emptyset \rightarrow \mathbb{R}$ satisfying $f() = c$, for any $c \in \mathbb{R}$, is an ADD. If two functions $f', f'' : \mathbb{B}^{I-i} \rightarrow \mathbb{R}$ are ADDs of binary vectors $(x_{i+1}, \dots, x_I)$, then the function $f : \mathbb{B}^{I-i+1} \rightarrow \mathbb{R}$ satisfying $f(x_i, \dots, x_I) = x_i \cdot f'(x_{i+1}, \dots, x_I) + (1 - x_i) \cdot f''(x_{i+1}, \dots, x_I)$ is an ADD of binary vectors $(x_i, \dots, x_I)$. For each sub-state $n \in [N]$ and each possible value $s'_n \in \mathcal{S}_n$ of that sub-state, the associated one-step transition probabilities $p_n(s'_n | s, a)$ can be represented as a collection of ADDs of the state s , one for each action $a \in \mathcal{A}$. Likewise, we can exploit context-specific independence in the rewards $r(s, a)$ by constructing a separate ADD for each reward component r_j , $j \in [J]$, and each action $a \in \mathcal{A}$. To facilitate an efficient representation, the order in which the components of s are inspected can vary across different transition probabilities and reward components. The size of an ADD's description scales with the number of functions employed in its definition.

Finally, a *rule system* represents a function $f : \mathcal{S} \rightarrow \mathbb{R}$ via a number of mutually exclusive and jointly exhaustive conjunctions that involve (negations of) some of the components of the state s , together with the associated function values $f(s)$ if the respective conjunctions are satisfied. We can exploit context-specific independence in the transition probabilities $p_n(s'_n | s, a)$ by constructing a separate rule system for each sub-state $n \in [N]$, each possible value $s'_n \in \mathcal{S}_n$ of that sub-state, and each action $a \in \mathcal{A}$. Likewise, we can exploit context-specific independence in the rewards $r(s, a)$ by constructing a separate rule system for each reward component r_j , $j \in [J]$, and each action $a \in \mathcal{A}$. The size of a rule system's description scales with the number of conjunctions employed in its definition.

Figure 8 provides representations of the transition probabilities $p_n(s'_{n,i} = 1 | s, a)$ from Example 5 as trees, ADDs and rule systems.



(a) Tree representation

$$\begin{aligned}
f^1(s_{n-1,1}, \dots, s_{n,i+1}) &= s_{n-1,1} \cdot g^1(s_{n,i}, s_{n,i+1}) + (1 - s_{n-1,1}) \cdot f^2(s_{n-1,2}, \dots, s_{n,i+1}) \\
f^2(s_{n-1,2}, \dots, s_{n,i+1}) &= s_{n-1,2} \cdot g^1(s_{n,i}, s_{n,i+1}) + (1 - s_{n-1,2}) \cdot f^3(s_{n-1,3}, \dots, s_{n,i+1}) \\
f^3(s_{n-1,3}, \dots, s_{n,i+1}) &= s_{n-1,3} \cdot g^1(s_{n,i}, s_{n,i+1}) + (1 - s_{n-1,3}) \cdot h^1(s_{n,i}, s_{n,i+1}) \\
g^1(s_{n,i}, s_{n,i+1}) &= s_{n,i} \cdot p_1 + (1 - s_{n,i}) \cdot g^2(s_{n,i+1}) \\
g^2(s_{n,i+1}) &= s_{n,i+1} \cdot p_2 + (1 - s_{n,i+1}) \cdot 0 \\
h^1(s_{n,i}, s_{n,i+1}) &= s_{n,i} \cdot q_1 + (1 - s_{n,i}) \cdot h^2(s_{n,i+1}) \\
h^2(s_{n,i+1}) &= s_{n,i+1} \cdot q_2 + (1 - s_{n,i+1}) \cdot 0
\end{aligned}$$

(b) ADD representation

$$\begin{aligned}
s_{n-1,1} = 1 \wedge s_{n,i} = 1 &\Rightarrow p_1; & s_{n-1,1} = 1 \wedge s_{n,i+1} = 1 &\Rightarrow p_2 \\
s_{n-1,2} = 1 \wedge s_{n,i} = 1 &\Rightarrow p_1; & s_{n-1,2} = 1 \wedge s_{n,i+1} = 1 &\Rightarrow p_2 \\
s_{n-1,3} = 1 \wedge s_{n,i} = 1 &\Rightarrow p_1; & s_{n-1,3} = 1 \wedge s_{n,i+1} = 1 &\Rightarrow p_2 \\
s_{n-1,1} = 0 \wedge s_{n-1,2} = 0 \wedge s_{n-1,3} = 0 \wedge s_{n,i} = 1 &\Rightarrow q_1 \\
s_{n-1,1} = 0 \wedge s_{n-1,2} = 0 \wedge s_{n-1,3} = 0 \wedge s_{n,i+1} = 1 &\Rightarrow q_2 \\
s_{n,i} = 0 \wedge s_{n,i+1} = 0 &\Rightarrow 0
\end{aligned}$$

(c) Rule-based representation

Figure 8. Different representations of the transition probabilities $p_n(s_{n,i} = 1 | s, a)$, $n > 1$, $i \in [9]$ and $a_n = 0$, for the FMDP from Example 5. In part (b), we assume that $p_n(s_{n,i} = 1 | s, a) = f^1(s)$.

In contrast to trees, ADDs and rule systems, MILP-based feature representations can exploit parsimony in both the state and the action space.

Example 7 (Predictive Maintenance, Cont'd). *Consider a variant of our maintenance problem that comprises N machines in total, out of which m should be repaired in each period. In this case,*

the action space satisfies $|\mathcal{A}| = \binom{N}{m}$. Since the representations of trees, ADDs and rule systems grow linearly in the number of actions, they grow exponentially in a natural problem description: for $N = 100$ machines and $m = 20$ repairs per period, for example, we have approximately $5.4 \cdot 10^{20}$ actions. In contrast, for each transition probability $p_n(s'_{n,i} = 1 | s, a)$ the MILP-based feature representation of Example 5 can readily accommodate this setting through the inclusion of a single additional feature $\phi_4(s, a) = \mathbf{1}[a_n = 1]$ for each machine $n \in [N]$.

Recall that our MILP-based features can be naturally represented in size $F_c \cdot (F + F_l + F_b)$, the number of feature constraints times the number of features and auxiliary (continuous and binary) variables. We next show these MILP-based feature representations are at least as efficient and sometimes significantly more efficient than trees, ADDs and rule systems in characterizing context-specific independence. To see this, consider a variant of our maintenance problem where the transition probabilities of each machine $n > 1$ depend on whether or not at least 50% of its predecessor machines are in states 8, 9 or 10. In this case, any representation of the transition probabilities via trees or rule systems would necessarily grow exponentially in the problem description since every subset of at least half of each machine's predecessors needs to be covered either by a path from the root node to a leaf (in the case of trees) or a probability rule (in the case of rule systems). In contrast, ADDs as well as MILP-based feature representations can handle this variant efficiently, the latter through the inclusion of $n - 1$ features $\phi_n(s, a) = \mathbf{1}[\sum_{i \in [n]} (s_{i,8} + s_{i,9} + s_{i,10}) \geq n/2]$, $n \in [N - 1]$. More formally, we can establish that *any* of the three representations from the literature is dominated by our MILP-based feature representation in the following sense.

Theorem 5 (Complexity of Representing Context-Specific Independence).

- (i) *For any description of a function $f : \mathbb{B}^I \rightarrow \mathbb{R}$ as a tree, ADD or rule system, there is an equivalent description as an MILP-based feature representation whose size is polynomially bounded in the original description.*
- (ii) *There are descriptions of functions $f : \mathbb{B}^I \rightarrow \mathbb{R}$ as MILP-based feature representations for which any equivalent description as a tree, ADD or rule system requires a size that is exponential in the original description.*

E-Companion B: Proofs

Proof of Theorem 1. In view of the first assertion, fix any feature map $\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{B}^F$ and define $\Phi_l = \{(s, a, \varphi_l) \in \mathcal{S} \times \mathcal{A} \times \mathbb{B} : \phi_l(s, a) = \varphi_l\}$ as the graph of the l -th component of ϕ . We then note that for all $(s, a) \in \mathcal{S} \times \mathcal{A}$ and $l \in [F]$, we have that

$$\phi_l(s, a) = \varphi_l \iff (s, a, \varphi_l) \in \Phi_l \iff (s, a, \varphi_l) \neq (s', a', \varphi'_l) \quad \forall (s', a', \varphi'_l) \in [\mathcal{S} \times \mathcal{A} \times \mathbb{B}] \setminus \Phi_l.$$

The latter condition holds precisely when

$$\begin{aligned} & [s \neq s' \vee a \neq a' \vee \varphi_l \neq \varphi'_l] && \forall (s', a', \varphi'_l) \in [\mathcal{S} \times \mathcal{A} \times \mathbb{B}] \setminus \Phi_l \\ \iff & \left[\bigvee_{n \in [N]} \bigvee_{i \in [S_n]} s_{ni} \neq s'_{ni} \vee \bigvee_{m \in [A]} a_m \neq a'_m \vee \varphi_l \neq \varphi'_l \right] && \forall (s', a', \varphi'_l) \in [\mathcal{S} \times \mathcal{A} \times \mathbb{B}] \setminus \Phi_l \\ \iff & \left[\sum_{n \in [N]} \sum_{i \in [S_n]} \frac{s'_{ni} - s_{ni}}{2s'_{ni} - 1} \geq 1 \vee \sum_{m \in [A]} \frac{a'_m - a_m}{2a'_m - 1} \geq 1 \vee \frac{\varphi'_l - \varphi_l}{2\varphi'_l - 1} \geq 1 \right] && \forall (s', a', \varphi'_l) \in [\mathcal{S} \times \mathcal{A} \times \mathbb{B}] \setminus \Phi_l \\ \iff & \sum_{n \in [N]} \sum_{i \in [S_n]} \frac{s'_{ni} - s_{ni}}{2s'_{ni} - 1} + \sum_{m \in [A]} \frac{a'_m - a_m}{2a'_m - 1} + \frac{\varphi'_l - \varphi_l}{2\varphi'_l - 1} \geq 1 && \forall (s', a', \varphi'_l) \in [\mathcal{S} \times \mathcal{A} \times \mathbb{B}] \setminus \Phi_l. \end{aligned}$$

In summary, for each $l \in [F]$ the constraint $\phi_l(s, a) = \varphi_l$ can be represented by a system of at most $2 \cdot |\mathcal{S} \times \mathcal{A}|$ linear constraints, together with the bounds $0 \leq \varphi_l \leq 1$, and with all coefficients in $\{-1, 0, 1\}$. The encoding size of this representation is therefore polynomial in F , $|\mathcal{S} \times \mathcal{A}|$, and $\sum_{n \in [N]} S_n + A$, which proves the first assertion.

As for the second assertion, we reduce from 3SAT, which is known to be NP-hard (Garey and Johnson, 1979). Given a 3SAT instance encoded in d bits, we construct an FMDP with $|\mathcal{A}| = 1$ and $\mathcal{S} = \{0, 1\}^d$, so that each state encodes a candidate instance and $\log |\mathcal{S}| = d$. The remaining components of the FMDP (q, p, r, γ) are not relevant for our construction and are thus omitted. We choose the feature ϕ to evaluate the satisfiability of the 3SAT instance encoded by each state, that is, $\phi(s, a)$ evaluates to 1 if and only if the instance encoded by s is satisfiable.

Suppose that ϕ admits a linear representation without auxiliary variables and with total encoding size L . Then ϕ can be evaluated by solving a linear program with input size L , which takes time $\text{poly}(L)$. If L were polynomial in $d = \log |\mathcal{S}|$, this would yield a polynomial-time algorithm for 3SAT, contradicting $P \neq NP$. Thus, the encoding size L must be super-polynomial in $\log |\mathcal{S}|$. $\square$

Proof of Proposition 1. In view of the first assertion, note that

$$\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n] = \sum_{s'_n \in \mathcal{S}'_n} \mathbf{1}[s_n = s'_n],$$

and that each indicator function $\mathbf{1}[s_n = s'_n]$ can be expressed by a variable $\zeta_{s'_n} \in \mathbb{R}_+$ satisfying

$$1 + \sum_{j \in [S_n]} \frac{s_{n,j} - s'_{n,j}}{2s'_{n,j} - 1} \leq \zeta_{s'_n} \leq \min_{j \in [S_n]} \left\{ \frac{s'_{n,j} + s_{n,j} - 1}{2s'_{n,j} - 1} \right\} \quad \forall s'_n \in \mathcal{S}'_n.$$

This representation has $F_l = |\mathcal{S}'_n|$ auxiliary variables and $F_c = 1 + |\mathcal{S}'_n| + S_n \cdot |\mathcal{S}'_n|$ constraints.

As for the second assertion, note that

$$\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n \quad \forall n \in \mathcal{N}] \iff \left(\sum_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n] \right) - (|\mathcal{N}| - 1) \leq \phi(s, a) \leq \min_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n],$$

and that each indicator function $\mathbf{1}[s_n \in \mathcal{S}'_n]$ can be represented using $|\mathcal{S}'_n|$ auxiliary variables and $(1 + S_n) \cdot |\mathcal{S}'_n|$ constraints (*cf.* Assertion 1). The overall representation thus has $F_l = \sum_{n \in \mathcal{N}} |\mathcal{S}'_n|$ auxiliary variables and $F_c = \sum_{n \in \mathcal{N}} (1 + S_n) \cdot |\mathcal{S}'_n| + 1 + |\mathcal{N}|$ constraints as claimed.

In view of the third assertion, note that

$$\phi(s, a) = \mathbf{1}[\exists n \in \mathcal{N} : s_n \in \mathcal{S}'_n] = 1 - \mathbf{1}[s_n \in \mathcal{S}_n \setminus \mathcal{S}'_n \quad \forall n \in \mathcal{N}],$$

and we can thus reuse the representation from Assertion 2.

As for the fourth assertion, note that

$$\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n \text{ for at least } \nu \text{ different } n \in \mathcal{N}] \iff \phi(s, a) = \mathbf{1} \left[\sum_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n] \geq \nu \right],$$

and that the latter equation holds if and only if $\phi(s, a) = \eta$ for the auxiliary variable $\eta \in \mathbb{B}$ satisfying

$$\frac{\left(\sum_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n] \right) - (\nu - 1)}{|\mathcal{N}| + 1} \leq \eta \leq 1 + \frac{\left(\sum_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n] \right) - \nu}{|\mathcal{N}|}.$$

Since each indicator function $\mathbf{1}[s_n \in \mathcal{S}'_n]$ can be represented using $|\mathcal{S}'_n|$ auxiliary variables and

$(1 + S_n) \cdot |\mathcal{S}'_n|$ constraints (*cf.* Assertion 1), the overall representation has $F_l = \sum_{n \in \mathcal{N}} |\mathcal{S}'_n|$ and $F_b = 1$ auxiliary variables as well as $F_c = \sum_{n \in \mathcal{N}} (1 + S_n) \cdot |\mathcal{S}'_n| + 2$ constraints.

In view of the last assertion, finally, note that

$$\phi(s, a) = \mathbf{1}[s_n \in \mathcal{S}'_n \text{ for at most } \nu \text{ different } n \in \mathcal{N}] \iff \phi(s, a) = \mathbf{1} \left[\sum_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n] \leq \nu \right],$$

and that the latter equation holds if and only if $\phi(s, a) = \eta$ for the auxiliary variable $\eta \in \mathbb{B}$ satisfying

$$\frac{\nu - \left(\sum_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n] - 1 \right)}{|\mathcal{N}| + 1} \leq \eta \leq 1 + \frac{\nu - \sum_{n \in \mathcal{N}} \mathbf{1}[s_n \in \mathcal{S}'_n]}{|\mathcal{N}|}.$$

Since each indicator function $\mathbf{1}[s_n \in \mathcal{S}'_n]$ can be represented using $|\mathcal{S}'_n|$ auxiliary variables and $(1 + S_n) \cdot |\mathcal{S}'_n|$ constraints (*cf.* Assertion 1), the overall representation has $F_l = \sum_{n \in \mathcal{N}} |\mathcal{S}'_n|$ and $F_b = 1$ auxiliary variables as well as $F_c = \sum_{n \in \mathcal{N}} (1 + S_n) \cdot |\mathcal{S}'_n| + 2$ constraints. $\square$

Similar to Theorem 1 of Littman (1997), our proof of Theorem 2 relies on a reduction to a combinatorial game proposed by Stockmeyer and Chandra (1979). In contrast to our setting, however, Littman (1997) studies propositional planning problems that are structurally different from the FMDPs considered in this paper, and as a result our reduction, while relying on the same combinatorial game, requires a different proof strategy.

Proof of Theorem 2. Since the feature map ϕ is assumed to be the identity, we will use Definition 1 (relating to FMDPs) rather than Definition 2 (relating to feature-based FMDPs) throughout this proof. To see that computing the optimal expected total discounted reward of an FMDP is EXPTIME-hard, we consider the following game proposed by Stockmeyer and Chandra (1979):

TWO-PERSON COMBINATORIAL GAME G_4 .

Instance. Given a 13-DNF formula² $F : \mathbb{B}^k \times \mathbb{B}^k \rightarrow \mathbb{B}$ and a starting position $(x^0, y^0) \in \mathbb{B}^k \times \mathbb{B}^k$. Player 1 takes the first turn.

Game. The players take turns in switching at most one of their variables $\{x_i\}_{i=1}^k$ (player 1) and $\{y_i\}_{i=1}^k$ (player 2). The player whose switch causes $F(x, y)$ to evaluate to 1 wins.

Question. Does the starting position admit a winning strategy for player 1, that is, a strategy under which player 1 always wins, no matter what player 2 does?

Game G_4 is known to be EXPTIME-complete (Stockmeyer and Chandra, 1979, Theorem 3.1).

Without loss of generality, we can assume that each player switches exactly one variable in each turn. Indeed, the possibility to not switch any variable in a particular move can be catered for by adding an auxiliary variable that does not impact the value of F . Moreover, we will use the fact that if there is a winning strategy for player 1, then there is a winning strategy under which the player always wins in at most $2^{\mathcal{O}(k)}$ steps (Stockmeyer and Chandra, 1979, p. 160).

For a given instance of G_4 , we first construct a simplified FMDP with scope $\mathcal{O}(k)$. We then outline how the FMDP can be modified so that it has scope $\mathcal{O}(\log k)$.

State Space. The state space is $\mathcal{S} = \mathbb{B}^2 \times \mathbb{B}^k \times \mathbb{B}^k \times \mathbb{B}^{\lceil \log_2 T \rceil}$. A state $(w, x, y, z) \in \mathcal{S}$ records the status w of the game ($(0, 0) = \text{open}$, $(1, 0) = \text{player 1 won}$, $(0, 1) = \text{player 2 won}$), the current position (x, y) as well as the number z of completed turns (in binary representation).

Action Space. We set $\mathcal{A} = \{e_i : i \in [k]\} \subseteq \mathbb{B}^k$, where $a \in \mathcal{A}$ records for each variable $i \in [k]$ whether x_i is being flipped (if $a_i = 1$) or not (if $a_i = 0$) by player 1.

Initial Distribution. The initial state is (w^0, x^0, y^0, z^0) with $w^0 = (0, 0)$ and $z^0 = (0, \dots, 0)$.

Transition Probabilities. If the game status is $(1, 0)$ or $(0, 1)$, then the FMDP remains at the current state with probability 1, independent of the selected action. If the game status is $(0, 0)$, on the other hand, then the position x of player 1 transitions deterministically to $x'_i = x_i + (1 - 2x_i) \cdot a_i$, $i \in [k]$, where $a \in \mathcal{A}$ is the current action. The position y of player 2 transitions to each of the states $y' = y + (1 - 2y_i) \cdot e_i$, $i \in [k]$, with probability $1/k$ each. The turn counter z is increased by one (in binary representation). The game status w , finally, is set to $w' = (1, 0)$ if $F(x', y) = 1$, it is set to $w' = (0, 1)$ if $F(x, y) = 1$ or $z = (1, \dots, 1)$, and it remains $w' = w$ otherwise. Note that the game status is backward looking with respect to the move of player 2, that is, a winning move of

²13-DNF: $C_1 \vee \dots \vee C_m$, $m \in \mathbb{N}$, where each C_i is a conjunction of at most 13 variables (or their negations).

player 2 in the z -th turn is only recognized by the new game status w' at the end of turn $z + 1$.

Reward Function. We set $r((w, x, y, z), a) = -1$ if $w = (0, 1)$ and $r((w, x, y, z), a) = 0$ otherwise.

Discount Factor. We choose any discount factor $\gamma \in (0, 1)$.

We claim that the optimal expected total discounted reward of the above FMDP vanishes if and only if player 1 has a winning strategy in game G_4 . Indeed, one readily verifies that the FMDP tracks the game's evolution in a canonical way. Moreover, if player 1 has a winning strategy, then player 2 cannot win the game, which implies that no non-zero rewards are earned under the winning strategy, thus resulting in zero expected total discounted reward. If there is a policy earning zero expected total discounted reward, finally, then it corresponds to a strategy under which player 1 wins in no more than T turns, irrespective of the moves of player 2 – this is, by definition, a winning strategy.

The FMDP described above can be implemented with a scope of $\mathcal{O}(k)$, that is, a scope that does not depend on the duration T of the game. Indeed, through a judicious choice of features, the transition probabilities for each position x'_i of player 1, $i \in [k]$, can be implemented with a scope of 4 as they depend on the game status w , the previous position x_i as well as the action a_i selected for position x_i . The transition probabilities for each position y'_i of player 2, $i \in [k]$, on the other hand, require a scope of $k + 2$ as they depend on the game status w as well as all previous positions $\{y_j\}_{j=1}^k$; this is needed to ensure that exactly one position is switched. The transition probabilities for each bit z'_i of the turn counter, $i \in [\lceil \log_2 T \rceil]$, as well as the status w' of the game, finally, can be implemented with a scope of 2 since they evolve deterministically.

The above FMDP can be modified so that it has a scope of $\mathcal{O}(\log k)$. To this end, we augment the state space to $\mathcal{S} = \mathbb{B}^2 \times \mathbb{B}^k \times \mathbb{B}^k \times \mathbb{B}^{\lceil \log_2 k \rceil} \times \mathbb{B}^{\lceil \log_2 T \rceil}$. A state (w, x, y, u, z) now additionally records in the game status w whose turn it is ($(0, 0) =$ player 1's turn, $(0, 1) =$ player 2's turn, $(1, 0) =$ player 1 has won, $(1, 1) =$ player 2 has won) as well as, if player 2 moves next, which of her variables y_j , $j = \sum_{l=1}^{\lceil \log_2 k \rceil} 2^{l-1} \cdot u_l$, is switched. In that case, the scope of the transition probabilities for each position y'_j of player 2, $j \in [k]$, can be reduced to $\log(k)$ since they become deterministic functions of the components of u , while the transition probabilities of u have scope 0 since they do not depend on any sub-state or action. For the sake of brevity, we omit the details of this tedious but otherwise straightforward extension. $\square$

Proof of Proposition 2. Replacing the value function v in (2) with our basis function approximation results in the following LP:

$$\begin{aligned}
& \underset{w}{\text{minimize}} && \sum_{s \in \mathcal{S}} \left[\prod_{n \in [N]} q_n(s_n) \right] \left[\sum_{k \in [K]} w_k \cdot \nu_k(s) \right] \\
& \text{subject to} && \sum_{k \in [K]} w_k \cdot \nu_k(s) \geq \sum_{j \in [J]} r_j(\phi(s, a)) + \gamma \sum_{s' \in \mathcal{S}} \left[\prod_{n \in [N]} p_n(s'_n | \phi(s, a)) \right] \left[\sum_{k \in [K]} w_k \cdot \nu_k(s') \right] \\
& && \forall (s, a) \in \mathcal{S} \times \mathcal{A} \\
& && w \in \mathbb{R}^K.
\end{aligned}$$

In view of the objective function of this problem, we note that

$$\begin{aligned}
& \sum_{s \in \mathcal{S}} \left[\prod_{n \in [N]} q_n(s_n) \right] \left[\sum_{k \in [K]} w_k \cdot \nu_k(s) \right] = \sum_{k \in [K]} w_k \sum_{s \in \mathcal{S}} \nu_k(s) \cdot \left[\prod_{n \in [N]} q_n(s_n) \right] \\
& = \sum_{k \in [K]} w_k \sum_{\substack{s_n \in \mathcal{S}_n: \\ n \in \bar{\mathfrak{s}}[\nu_k]}} \sum_{\substack{s_{n'} \in \mathcal{S}_{n'}: \\ n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]}} \nu_k(s) \cdot \left[\prod_{n \in \bar{\mathfrak{s}}[\nu_k]} q_n(s_n) \right] \cdot \left[\prod_{n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]} q_{n'}(s_{n'}) \right] \\
& = \sum_{k \in [K]} w_k \sum_{s \in \bar{\mathfrak{S}}[\nu_k]} \sum_{\substack{s'_{n'} \in \mathcal{S}_{n'}: \\ n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]}} \nu_k(s) \cdot \left[\prod_{n \in \bar{\mathfrak{s}}[\nu_k]} q_n(s_n) \right] \cdot \left[\prod_{n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]} q_{n'}(s'_{n'}) \right], \tag{8}
\end{aligned}$$

where the first identity applies standard algebraic manipulations, the second identity decomposes each state $s \in \mathcal{S}$ into its components s_n , $n \in \bar{\mathfrak{s}}[\nu_k]$, and $s_{n'}$, $n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]$, and the third identity utilizes our definition of $\bar{\mathfrak{S}}[\nu_k]$ and the fact that ν_k does not depend on the components $s_{n'}$, $n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]$. We next observe that (8) can be re-expressed as

$$\begin{aligned}
& \sum_{k \in [K]} w_k \left(\sum_{s \in \bar{\mathfrak{S}}[\nu_k]} \nu_k(s) \cdot \left[\prod_{n \in \bar{\mathfrak{s}}[\nu_k]} q_n(s_n) \right] \right) \cdot \left(\sum_{\substack{s'_{n'} \in \mathcal{S}_{n'}: \\ n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]}} \prod_{n' \in [N] \setminus \bar{\mathfrak{s}}[\nu_k]} q_{n'}(s'_{n'}) \right) \\
& = \sum_{k \in [K]} w_k \left(\sum_{s \in \bar{\mathfrak{S}}[\nu_k]} \nu_k(s) \cdot \left[\prod_{n \in \bar{\mathfrak{s}}[\nu_k]} q_n(s_n) \right] \right),
\end{aligned}$$

where the first row uses the distributive property of multiplication over addition, and the identity

holds since the second multiplier in the first row evaluates to one. Similar arguments show that

$$\sum_{s' \in \mathcal{S}} \left[\prod_{n \in [N]} p_n(s'_n | \phi(s, a)) \right] \left[\sum_{k \in [K]} w_k \cdot \nu_k(s') \right] = \sum_{k \in [K]} w_k \left(\sum_{s' \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s') \left[\prod_{n \in \bar{\mathcal{S}}[\nu_k]} p_n(s'_n | \phi(s, a)) \right] \right),$$

and we thus conclude that under our value function approximation, (2) indeed simplifies to (3).

To prove the second part, we show that Problem (3) is feasible and not unbounded. The statement then follows from the fact that every feasible finite-dimensional LP attains its optimal value if the latter is finite. To see that (3) is feasible, fix $C = (1 - \gamma)^{-1} \cdot \max\{r(\phi(s, a)) : (s, a) \in \mathcal{S} \times \mathcal{A}\}$ and let $w^1 \in \mathbb{R}^K$ be such that $\sum_{k \in [K]} w_k^1 \cdot \nu_k(s) = 1$ for all $s \in \mathcal{S}$. We then observe that

$$\begin{aligned} \sum_{k \in [K]} (C \cdot w_k^1) \cdot \nu_k(s) &= C = \frac{1}{1 - \gamma} \cdot \max\{r(\phi(s', a)) : (s', a) \in \mathcal{S} \times \mathcal{A}\} \\ &= \max\{r(\phi(s', a)) : (s', a) \in \mathcal{S} \times \mathcal{A}\} + \gamma \cdot C \\ &\geq \max_{a \in \mathcal{A}} \left[r(\phi(s, a)) + \gamma \sum_{k \in [K]} (C \cdot w_k^1) \sum_{s' \in \bar{\mathcal{S}}[\nu_k]} \nu_k(s') \prod_{n \in \bar{\mathcal{S}}[\nu_k]} p_n(s'_n | \phi(s, a)) \right] \end{aligned}$$

for all $s \in \mathcal{S}$, where the first two identities are due to the definitions of w^1 and C , respectively, the third identity applies basic algebraic manipulations, and the inequality follows from the definition of w^1 , the fact that $\max\{r(\phi(s, a)) : a \in \mathcal{A}\} \leq \max\{r(\phi(s', a)) : (s', a) \in \mathcal{S} \times \mathcal{A}\}$, as well as the inequality $\sum_{s' \in \bar{\mathcal{S}}[\nu_k]} \prod_{n \in \bar{\mathcal{S}}[\nu_k]} p_n(s'_n | \phi(s, a)) \leq 1$, which holds for all $(s, a) \in \mathcal{S} \times \mathcal{A}$.

To see that Problem (3) is not unbounded, finally, note that Problem (2) bounds (3) from below. As we have discussed in Section 4, however, the optimal value of (2) coincides with the expected total discounted reward of an optimal policy to the FMDP, which is bounded by construction. $\square$

Proof of Theorem 3. To see that Algorithm 1 terminates in finite time, we note that by construction of the algorithm, every constraint of Problem (3) is added at most once. The claim now follows from the fact that Problem (3) comprises finitely many constraints.

To see that $\hat{w}$ is feasible in Problem (3), fix any $(s, a) \in \mathcal{S} \times \mathcal{A}$ and observe that

$$\begin{aligned}
\sum_{k \in [K]} \hat{w}_k \cdot \nu_k(s) &= \left[\sum_{k \in [K]} w_k^\epsilon \cdot \nu_k(s) \right] + \frac{\epsilon}{1-\gamma} \\
&\geq r(\phi(s, a)) + \gamma \sum_{s' \in \mathcal{S}} p(s' | \phi(s, a)) \left[\sum_{k \in [K]} w_k^\epsilon \cdot \nu_k(s') \right] - \epsilon + \frac{\epsilon}{1-\gamma} \\
&= r(\phi(s, a)) + \gamma \sum_{s' \in \mathcal{S}} p(s' | \phi(s, a)) \left[\left(\sum_{k \in [K]} w_k^\epsilon \cdot \nu_k(s') \right) + \frac{\epsilon}{1-\gamma} \right],
\end{aligned}$$

where the first identity employs the definition of $\hat{w}$, the first inequality exploits the ϵ -feasibility of w^ϵ , and the second identity reorders terms and uses that $\sum_{s' \in \mathcal{S}} p(s' | \phi(s, a)) = 1$. Applying the identity from the first row again to the term inside the square bracket in the last row, we obtain

$$\sum_{k \in [K]} \hat{w}_k \cdot \nu_k(s) \geq r(\phi(s, a)) + \gamma \sum_{s' \in \mathcal{S}} p(s' | \phi(s, a)) \left[\sum_{k \in [K]} \hat{w}_k \cdot \nu_k(s) \right] \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A},$$

that is, $\hat{w}$ is indeed feasible in Problem (3). The feasibility of $\hat{w}$ in (3) and the fact that w^ϵ optimally solves a relaxation of (3) then implies that $F(w^\epsilon) \leq F^* \leq F(\hat{w})$.

The upper bound for $F(\hat{w})$ can be proved as follows:

$$F(\hat{w}) - F^* \leq F(\hat{w}) - F(w^\epsilon) = \sum_{s \in \mathcal{S}} q(s) \cdot \left[\sum_{k \in [K]} (\hat{w}_k - w_k^\epsilon) \cdot \nu_k(s) \right] = \frac{\epsilon}{1-\gamma},$$

where the first inequality holds since w^ϵ optimally solves the master problem of the last iteration of Algorithm 1, which itself is a relaxation of Problem (3), the first identity employs formulation (2) and Proposition 2 to evaluate $F(\hat{w})$ and $F(w^\epsilon)$, and the last identity follows from the definitions of $\hat{w}$ and w^1 as well as the fact that $\sum_{s \in \mathcal{S}} q(s) = 1$.

To see that $F(w^\epsilon) \geq F^* - \epsilon(1-\gamma)^{-1}$, we recall that $\hat{w} = w^\epsilon + \frac{\epsilon}{1-\gamma} \cdot w^1$. Using this and the linearity of the objective function F , we get

$$F(\hat{w}) = F(w^\epsilon) + \frac{\epsilon}{1-\gamma}.$$

The feasibility of $\hat{w}$ then yields the desired lower bound:

$$F(w^\epsilon) = F(\hat{w}) - \frac{\epsilon}{1-\gamma} \geq F^* - \frac{\epsilon}{1-\gamma}.$$

This completes the proof. $\square$

Proof of Theorem 4. As discussed in Algorithm 1, a maximally violated constraint can be extracted from an optimal solution to the optimization problem

$$\begin{aligned} & \underset{s, a, \varphi, \zeta}{\text{maximize}} && \sum_{j \in [J]} r_j(\varphi) + \gamma \sum_{k \in [K]} w_k^* \cdot \bar{\nu}_k(\varphi) - \sum_{k \in [K]} w_k^* \cdot \nu_k(s) \\ & \text{subject to} && (s, a) \in \mathcal{S} \times \mathcal{A}, \quad (\varphi, \zeta; s, a) \in \mathcal{F}, \quad \zeta \in \mathbb{R}^{F_l} \times \mathbb{B}^{F_b}. \end{aligned} \tag{9}$$

The reward components $r_j(\varphi)$, $j \in [J]$, in this problem can be reformulated as

$$r_j(\varphi) = \sum_{f \in \mathfrak{S}[r_j]} r_j(f) \cdot \mathbf{1}[\varphi_i = f_i \ \forall i \in \mathfrak{s}[r_j]] = \sum_{f \in \mathfrak{S}[r_j]} r_j(f) \cdot \eta_{jf}$$

for $\eta_{jf} = \mathbf{1}[\varphi_i = f_i \ \forall i \in \mathfrak{s}[r_j]]$, $j \in [J]$ and $f \in \mathfrak{S}[r_j]$. One readily verifies that $\eta_{jf} = \mathbf{1}[\varphi_i = f_i \ \forall i \in \mathfrak{s}[r_j]]$ if and only if $\eta_{jf} \in [0, 1]$ and

$$\eta_{jf} \leq (2f_i - 1)\varphi_i + 1 - f_i \ \forall i \in \mathfrak{s}[r_j] \quad \text{as well as} \quad \eta_{jf} \geq 1 + \sum_{i \in \mathfrak{s}[r_j]} \frac{\varphi_i - f_i}{2f_i - 1}.$$

Similar arguments show that

$$\bar{\nu}_k(\varphi) = \sum_{f \in \mathfrak{S}[\bar{\nu}_k]} \bar{\nu}_k(f) \cdot \mathbf{1}[\varphi_i = f_i \ \forall i \in \mathfrak{s}[\bar{\nu}_k]] = \sum_{f \in \mathfrak{S}[\bar{\nu}_k]} \bar{\nu}_k(f) \cdot \xi_{kf}$$

for $\xi_{kf} = \mathbf{1}[\varphi_i = f_i \ \forall i \in \mathfrak{s}[\bar{\nu}_k]]$, $k \in [K]$ and $f \in \mathfrak{S}[\bar{\nu}_k]$, which holds if and only if $\xi_{kf} \in [0, 1]$ and

$$\xi_{kf} \leq (2f_i - 1)\varphi_i + 1 - f_i \ \forall i \in \mathfrak{s}[\bar{\nu}_k] \quad \text{as well as} \quad \xi_{kf} \geq 1 + \sum_{i \in \mathfrak{s}[\bar{\nu}_k]} \frac{\varphi_i - f_i}{2f_i - 1},$$

as well as

$$\nu_k(s) = \sum_{s' \in \mathfrak{S}[\nu_k]} \nu_k(s') \cdot \mathbf{1}[[s]_i = [s']_i \ \forall i \in \mathfrak{s}[\nu_k]] = \sum_{s' \in \mathfrak{S}[\nu_k]} \nu_k(s') \cdot \beta_{ks'}$$

for $\beta_{ks'} = \mathbf{1}[[s]_i = [s']_i \ \forall i \in \mathfrak{s}[\nu_k]]$, $k \in [K]$ and $s' \in \mathfrak{S}[\nu_k]$, which holds if and only if $\beta_{ks'} \in [0, 1]$ and

$$\beta_{ks'} \leq (2[s']_i - 1)[s]_i + 1 - [s']_i \ \forall i \in \mathfrak{s}[\nu_k] \quad \text{as well as} \quad \beta_{ks'} \geq 1 + \sum_{i \in \mathfrak{s}[\nu_k]} \frac{[s]_i - [s']_i}{2[s']_i - 1}.$$

The statement now follows if we replace $r_j(\varphi)$, $\bar{\nu}_k(\varphi)$ and $\nu_k(s)$ in (9) with their reformulations. $\square$

Proof of Proposition 3. Theorem 4 determines a maximally violated constraint through the solution of an MILP whose size is polynomial in the description of the feature-based FMDP. We thus conclude that determining a maximally violated constraint is in NP. To see that the problem is also strongly NP-hard, consider the following integer feasibility problem:

0/1 INTEGER PROGRAMMING. Instance. Given are $G \in \mathbb{Z}^{L \times N}$ and $h \in \mathbb{Z}^L$. Question. Is there a vector $x \in \{0, 1\}^N$ such that $Gx \leq h$?

For a given instance of the integer feasibility problem, we construct a feature-based FMDP with the state space $\mathcal{S} = \times_{n \in [N]} \mathcal{S}_n$ with $\mathcal{S}_n = \mathbb{B}$, $n \in [N]$, the action space $\mathcal{A} = \mathbb{B}$, the feature map $\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{B}$ satisfying $\phi(s, a) = \mathbf{1}[Gs \leq h]$, the reward function satisfying $r(s, a) = \phi(s, a)$, any scope-1 initial distribution q , any scope-1 transition kernel p as well as any discount factor; we not specify those components further since they will not impact our argument. We set the basis function weights w to zero so that the choice of basis functions for the value function is irrelevant as well. In this case, any maximally violated constraint (s^*, a^*) simply maximizes $\phi(s, a)$ over $\mathcal{S} \times \mathcal{A}$, and we conclude that the maximum constraint violation is 1 precisely when the answer to the integer feasibility problem is affirmative. Since the integer feasibility problem is strongly NP-complete (Garey and Johnson, 1979), the statement of the proposition then follows.

It remains to be shown that the feature map $\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{B}$ admits a representation that is

polynomial in the length of the integer feasibility instance. To this end, we claim that

$$\phi(s, a) = \varphi \iff \left[\begin{array}{ll} g_l^\top s \geq h_l + \frac{1}{2} - M \cdot \zeta_l & \forall l \in [L] \\ g_l^\top s \leq h_l + M \cdot (1 - \zeta_l) & \forall l \in [L] \\ \varphi \leq \zeta_l & \forall l \in [L] \\ \varphi \geq \left(\sum_{l \in [L]} \zeta_l \right) - (L - 1) & \end{array} \right] \text{ for some } \zeta \in \mathbb{B}^L, \quad (10)$$

where $g_l^\top \in \mathbb{Z}^N$ is the l -th row of matrix G and $M = \max_{l \in [L]} \left[1 + |h_l| + \sum_{n \in [N]} |g_{ln}| \right]$. To see this, note that thanks to the integrality of G and h , the equation system (10) ensures that $\zeta_l = \mathbf{1}[g_l^\top s \leq h_l]$ for all $l \in [L]$ as well as $\varphi = \min\{\zeta_l : l \in [L]\}$. Thus, whenever $\phi(s, a) = 1$, that is, whenever $Gs \leq h$, we have $\zeta = \mathbf{1}$ and thus $\varphi = 1$. In contrast, whenever $\phi(s, a) = 0$, that is, whenever $Gs \not\leq h$, we have $\zeta_l = 0$ for at least one $l \in [L]$, and thus $\varphi = 0$ as desired. $\square$

Proof of Corollary 1. The statement follows immediately from the optimization problem in Theorem 4 if we fix the state s , disregard the last expression from the objective function and remove the auxiliary variables $\{\beta_{ks'}\}$. $\square$

Proof of Proposition 4. Let $F^\star$ denote the minimum cardinality of any feature set that exactly recovers p , and define

$$\delta = \left[\begin{array}{ll} \underset{\Gamma, \psi}{\text{minimize}} & \sum_{n \in [N]} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \|p_n(\cdot | s, a) - \psi_n(\phi_\Gamma(s, a))\|_2^2 \\ \text{subject to} & \Gamma \subseteq \Phi, \quad |\Gamma| < F^\star \quad \text{and} \quad \psi_n : \mathbb{B}^{|\Gamma|} \rightarrow \Delta(\mathcal{S}_n), \quad n \in [N] \end{array} \right]$$

as the minimum unregularized objective value of any feature selection Γ with less than $F^\star$ features in Problem (5'). We claim that the statement of the proposition holds for $\lambda_0 = \delta/F^\star$. We prove our claim by showing that (i) any feature selection with less than $F^\star$ features attains an objective value exceeding $\lambda \cdot F^\star$ in Problem (5'); (ii) any feature selection with $F^\star$ features that exactly recovers p achieves an objective value of exactly $\lambda \cdot F^\star$ in (5'); (iii) any feature selection with $F^\star$ features that does not exactly recover p , as well as any feature set with more than $F^\star$ features, attains an objective value of exceeding $\lambda \cdot F^\star$ in (5'); and (iv) any feature selection Γ attaining an objective value of

$\lambda \cdot F^\star$ in (5') must have $F^\star$ features and recover p exactly. The proof then concludes since $\delta > 0$, for otherwise there would be a feature set with cardinality less than $F^\star$ that exactly recovers p .

In view of claim (i), note that any feature set $\Gamma \subseteq \Phi$ with $|\Gamma| < F^\star$ must have an objective value of at least $\delta = \lambda_0 \cdot F^\star > \lambda \cdot F^\star$ by definition of δ . Claims (ii)–(iv) follow immediately from the construction of the objective function in Problem (5'). As for the last claim, note first that the claims (i) and (iii) imply that $|\Gamma| = F^\star$, for otherwise the objective value would have to exceed $\lambda \cdot F^\star$. In that case, however, the objective value of Γ satisfies

$$\sum_{n \in [N]} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \|p_n(\cdot | s, a) - \psi_n(\phi_\Gamma(s, a))\|_2^2 + \lambda \cdot F^\star = \lambda \cdot F^\star,$$

which immediately implies that ϕ_Γ recovers p exactly. $\square$

Proof of Observation 1. Note that the objective function in Problem (6) satisfies

$$\sum_{n \in [N]} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \|p_n(\cdot | s, a) - \psi_n(\phi_{\Gamma \cup \{\varphi\}}(s, a))\|_2^2 = \sum_{n \in [N]} \sum_{\phi \in \mathbb{B}^{|\Gamma|+1}} \sum_{(s,a) \in \Omega(\phi)} \|p_n(\cdot | s, a) - \psi_n(\phi)\|_2^2,$$

and that the right-hand side of this equation can be expressed as

$$\sum_{n \in [N]} \sum_{\phi \in \mathbb{B}^{|\Gamma|+1}} \left(|\Omega(\phi)| \cdot \left\| \frac{1}{|\Omega(\phi)|} \cdot \sum_{(s,a) \in \Omega(\phi)} p_n(\cdot | s, a) - \psi_n(\phi) \right\|_2^2 + c_n(\phi) \right), \quad (11)$$

where the term

$$c_n(\phi) = \sum_{(s,a) \in \Omega(\phi)} \|p_n(\cdot | s, a)\|_2^2 - \frac{1}{|\Omega(\phi)|} \cdot \left\| \sum_{(s,a) \in \Omega(\phi)} p_n(\cdot | s, a) \right\|_2^2$$

does not depend on ψ_n . The expression (11) has $\psi_n^\star$ from the statement of the observation as an unconstrained minimizer. Since $\psi_n^\star(\phi) \in \Delta(\mathcal{S}_n)$ for all $n \in [N]$ and $\phi \in \mathbb{B}^{|\Gamma|+1}$, it must also be the constrained minimizer. $\square$

Proof of Theorem 5. In view of statement (i), consider first a representation of the function

$f : \mathbb{B}^I \rightarrow \mathbb{R}$ as a rule system. Concretely, fix a representation of the form

$$\bigwedge_{(i,v) \in \mathfrak{A}_r} (x_i = v) \Rightarrow f(x) = c_r \quad \forall r \in \mathfrak{R},$$

where $\mathfrak{R}$ is the set of rules to determine f , $\mathfrak{A}_r$ is the set of antecedents (or premises) for rule r and c_r is the consequent (or conclusion) of r . This rule-based representation is equivalent to the MILP-based feature representation under which $f(x) = c_r$ if and only if $\phi_r(x) = 1$, $r \in \mathfrak{R}$, where the MILP-based features $\phi_r(x) = \mathbf{1}[\bigwedge_{(i,v) \in \mathfrak{A}_r} (x_i = v)]$ have the polynomial-size representation

$$\left. \begin{aligned} \phi_r(x) &\leq 1 - x_i \quad \forall (i, 0) \in \mathfrak{A}_r; \quad \phi_r(x) \leq x_i \quad \forall (i, 1) \in \mathfrak{A}_r \\ \phi_r(x) &\geq \sum_{(i,0) \in \mathfrak{A}_r} (1 - x_i) + \sum_{(i,1) \in \mathfrak{A}_r} x_i - [|\mathfrak{A}_r| - 1] \end{aligned} \right\} \quad \forall r \in \mathfrak{R}.$$

This concludes statement (i) for rule systems. Since every decision tree can be represented as a rule system with one rule for each path from the root node to one of the leaf nodes, our proof for rule systems immediately applies to decision tree representations as well.

To see that statement (i) also holds for ADDs, we first observe that the empty function $f : \emptyset \rightarrow \mathbb{R}$ satisfying $f() = c$ has a trivial MILP-based feature representation. Assume now that the two functions $f', f'' : \mathbb{B}^{I-i}$ are ADDs of binary vectors $(x_{i+1}, \dots, x_I)$ with polynomial-sized MILP-based feature representations $f'(x_{i+1}, \dots, x_I) = c'_r$ if $\phi'_r(x_{i+1}, \dots, x_I) = 1$, $r \in \mathfrak{R}'$, and $f''(x_{i+1}, \dots, x_I) = c''_r$ if $\phi''_r(x_{i+1}, \dots, x_I) = 1$, $r \in \mathfrak{R}''$. This implies that there are polyhedra $\mathcal{F}', \mathcal{F}''$ as well as auxiliary vectors $\zeta' \in \mathbb{R}^{F'_l} \times \mathbb{B}^{F'_b}$ and $\zeta'' \in \mathbb{R}^{F''_l} \times \mathbb{B}^{F''_b}$, all of polynomial size, such that $\phi'_r(x_{i+1}, \dots, x_I) = \varphi \in \mathbb{B}^{|\mathfrak{R}'|}$ precisely when $(\varphi, \zeta'; x_{i+1}, \dots, x_I) \in \mathcal{F}'$ and $\phi''_r(x_{i+1}, \dots, x_I) = \varphi \in \mathbb{B}^{|\mathfrak{R}''|}$ precisely when $(\varphi, \zeta''; x_{i+1}, \dots, x_I) \in \mathcal{F}''$, respectively. We show that in this case, the function $f : \mathbb{B}^{I-i+1}$ satisfying $f(x_i, \dots, x_I) = x_i \cdot f'(x_{i+1}, \dots, x_I) + (1 - x_i) \cdot f''(x_{i+1}, \dots, x_I)$ has an MILP-based feature representation $f(x_i, \dots, x_I) = c_r$ if $\phi_r(x_i, \dots, x_I) = 1$, $r \in \mathfrak{R}$, of polynomial size. Indeed, fix $\mathfrak{R} = \{(1, r') : r' \in \mathfrak{R}'\} \cup \{(0, r'') : r'' \in \mathfrak{R}''\}$, $c_r = c'_{r'}$ for $r = (1, r') \in \mathfrak{R}$, $r' \in \mathfrak{R}'$, and $c_r = c''_{r''}$ for $r = (0, r'') \in \mathfrak{R}$, $r'' \in \mathfrak{R}''$. The claim follows if we can show that the feature map ϕ satisfying

$$\begin{aligned} \phi_r(x_i, \dots, x_I) &= \mathbf{1}[x_i = 1 \text{ and } \phi'_{r'}(x_{i+1}, \dots, x_I) = 1] \quad \text{for } r = (1, r') \in \mathfrak{R}, r' \in \mathfrak{R}' \\ \text{and } \phi_r(x_i, \dots, x_I) &= \mathbf{1}[x_i = 0 \text{ and } \phi''_{r''}(x_{i+1}, \dots, x_I) = 1] \quad \text{for } r = (0, r'') \in \mathfrak{R}, r'' \in \mathfrak{R}'' \end{aligned}$$

has an MILP representation of polynomial size. This is the case since we have

$$\phi(x_1, \dots, x_I) = \varphi \in \mathbb{B}^{|\mathcal{R}|} \iff \begin{bmatrix} \varphi \geq \varphi' - (1 - x_i) \cdot e, & \varphi \leq \varphi' + (1 - x_i) \cdot e \\ \varphi \geq \varphi'' - x_i \cdot e, & \varphi \leq \varphi'' + x_i \cdot e \\ (\varphi', \zeta'; x_{i+1}, \dots, x_I) \in \mathcal{F}_{r'} \\ (\varphi'', \zeta''; x_{i+1}, \dots, x_I) \in \mathcal{F}_{r''} \end{bmatrix}.$$

As for statement (ii), consider first the parity function $f(x_1, \dots, x_I) = 1$ if $\sum_{i=1}^I x_i$ is even; $= 0$ otherwise. This function cannot be computed without considering the value x_i of each of its inputs $i \in [I]$, and thus any representation of this function as a tree or rule system necessarily scales exponentially in I . In contrast, we have $f(x_1, \dots, x_I) = 1$ if $\phi(x_1, \dots, x_I) = 1$; $= 0$ otherwise for the feature map $\phi(x_1, \dots, x_I) = \mathbf{1}[\sum_{i=1}^I x_i \text{ is even}]$, which has the MILP representation

$$\phi(x_1, \dots, x_I) = \varphi \iff \left[\begin{array}{l} \varphi = 1 - z_I, \quad z_1 = x_1 \\ z_{i+1} = z_i + x_{i+1} - 2y_{i+1} \\ y_{i+1} \leq z_i, \quad y_{i+1} \leq z_{i+1} \\ y_{i+1} \geq z_i + z_{i+1} - 1 \end{array} \right\} \quad \forall i \in [I-1],$$

$\varphi \in \mathbb{B}$, that is of polynomial size. Note that in this formulation, we have $y_{i+1} = \mathbf{1}[z_i = 1 \wedge z_{i+1} = 1]$ for all $i \in [I-1]$ and $z_i = \mathbf{1}[\sum_{i'=1}^i x_{i'} \text{ is odd}]$ for all $i \in [I]$.

The argument from the previous paragraph does not extend to ADDs; in fact, ADDs can also represent the parity function efficiently. To see that statement (ii) nevertheless extends to ADDs, consider the function $f(a_1, \dots, a_I; b_1, \dots, b_I)$ that computes one of the $2I$ bits of the result of multiplying the binary numbers $(a_1, \dots, a_I)$ and $(b_1, \dots, b_I)$. It follows from Bryant (1991) that ADDs cannot represent this function in polynomial size. In contrast, we show next that this function can be represented efficiently via MILP-based features. To this end, consider the following feature map, which computes the entire result of the multiplication:

$$\phi(a_1, \dots, a_I; b_1, \dots, b_I) = \varphi \iff \left[\begin{array}{l} \sum_{i \in [2I]} 2^{i-1} \cdot \varphi_i = \sum_{i,j \in [I]} 2^{i+j-2} \cdot \zeta_{ij} \\ \zeta_{ij} \leq a_i, \quad \zeta_{ij} \leq b_j, \quad \zeta_{ij} \geq a_i + b_j - 1 \quad \forall i, j \in [I] \end{array} \right]$$

To see that the above representation is correct, observe first that for all $i, j \in [I]$ we have $\zeta_{ij} = a_i \cdot b_j$.

The right-hand side of the first constraint therefore evaluates to

$$\sum_{i,j \in [I]} 2^{i+j-2} \cdot a_i \cdot b_j = \left(\sum_{i \in [I]} 2^{i-1} a_i \right) \left(\sum_{j \in [I]} 2^{j-1} b_j \right),$$

where $\sum_{i \in [I]} 2^{i-1} \cdot a_i$ and $\sum_{j \in [I]} 2^{j-1} \cdot b_j$ are the numbers represented by the binary vectors $(a_1, \dots, a_I)$ and $(b_1, \dots, b_I)$, respectively. The left-hand side of the first constraint thus ensures that the number represented by the binary vector $(\varphi_1, \dots, \varphi_{2I})$ equals the product of $(a_1, \dots, a_I)$ and $(b_1, \dots, b_I)$, as desired. Our constraints defining the feature map ϕ involve coefficients that are exponential in the length $2I$ of its input; this can be avoided by emulating a binary multiplier through MILP constraints. We omit the details of this tedious but otherwise straightforward representation. $\square$